A CAPSTONE PROJECT REPORT ON

SUBMITTED IN PARTIAL FULFILMENT FOR THE AWARD OF **THE DEGREE OF EMBA BUSINESS ANALYTICS (EMBA - BA)**

SUBMITTED BY:

HAJI RAYYAAN TAJ

ROLL NO: 2440

BATCH: 2024-26

UNDER THE GUIDANCE OF **Professor Nirmala Joshi**

MET's ASIAN MANAGEMENT AND DEVELOPMENT CENTRE BHUJBAL KNOWLEDGE CITY

BANDRA RECLAMATION, BANDRA (WEST),

MUMBAI-400 050

Acknowledgment

I would like to extend my heartfelt gratitude to **Professor Nirmala Joshi Ma'am** who contributed to the successful completion of this report. We are grateful to our mentors and advisors, whose guidance and insights have been invaluable in shaping our analysis and approach. Additionally, we would like to acknowledge the assistance provided by our colleagues and teammates, who supported us throughout the project. Their valuable feedback helped us refine our methodology and deepen our understanding of the dataset.

I likewise thank **Prof. Arun Patil**, our dean, and **Dr. Sangeeta Tandon**, Director, for their help and guidance, allowing me to embark upon this project. Concluding, I would like to thank every one of them with whom I collaborated. Their due participation and inspiration made a successful landing for this project.

Regards,
Haji Rayyaan Taj
Date:
Place: Mumbai

INDEX

Forecasting and Analysis Of NIFTY 50

1. Introduction

This report provides an in-depth analysis of the NIFTY 50 index over the past 10 years (from April 2010 to March 2020). By combining data visualization, statistical summaries, and machine learning techniques, the project aims to uncover market trends, assess volatility, and forecast future price movements. The insights gained from this analysis are intended to support data-driven decision-making for investors and analysts.

The study focuses on several key aspects:

1. **Data Loading and Initial Exploration**
    - **Objective:** Understand the basic structure and quality of the historical NIFTY 50 data.
    - **Approach:** The data is imported using Python's pandas library, and initial checks (like viewing data dimensions, checking for missing values, and summarizing statistics) are performed. This step provides a clear picture of the dataset, including the number of records, the range of values, and the presence of any inconsistencies.
2. **Technical Indicator Calculation**
    - **Objective:** Smooth out short-term fluctuations and reveal long-term trends in the NIFTY 50 index.
    - **Approach:** Key indicators such as moving averages (7-day, 30-day, and 100-day) and daily percentage changes are calculated. These indicators help in identifying patterns, such as support and resistance levels, and in measuring market volatility.
3. **Visualizing Market Trends**
    - **Objective:** Present the data visually to understand market behavior over time.
    - **Approach:** Various plots are generated:
        - **Line Charts:** Compare the actual index price with the calculated moving averages.
        - **Histograms:** Show the distribution of daily returns, highlighting market volatility.
        - **Scatter Plots:** Identify significant drops (or "crashes") in the market by plotting dates with extreme negative returns.

- These visualizations provide a clear and accessible way to see how the market has evolved and to spot periods of high volatility or significant trend changes.

4. **Forecasting Future Prices Using Machine Learning**
    - **Objective:** Predict future movements of the NIFTY 50 index using historical data.
    - **Approach:** The Facebook Prophet model is used for forecasting. The model is trained on recent data and extended to predict future values for the next three years. Along with point forecasts, the model provides confidence intervals that illustrate the uncertainty in long-term predictions. This forecasting step helps in understanding potential future trends and preparing for market uncertainties.
5. **Model Evaluation and Error Analysis**
    - **Objective:** Assess the accuracy of the forecasts.
    - **Approach:** Standard error metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE) are computed. Residual analysis is also performed through various plots, which helps identify where the model's predictions differ significantly from actual values. This evaluation is critical to understand the reliability of the forecasting model and to guide further refinements.
6. **Correlation and Feature Analysis**
    - **Objective:** Discover relationships between various financial indicators and the NIFTY 50 index.
    - **Approach:** A correlation matrix is generated to examine how different variables (such as opening price, closing price, high, low, and other technical indicators) relate to each other. This step is essential for identifying which features are most influential in driving market trends.

Conclusion

The analysis provides a detailed look at the historical performance of the NIFTY 50 index, uncovers key trends and patterns, and offers a forecast of future price movements. The insights from moving averages, volatility analysis, and the forecasting model can help investors make informed decisions. Additionally, the evaluation of error metrics and residuals identifies areas for model improvement, setting the stage for further research and refinement.

INPUT 1:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import yfinance as yf
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error, mean_squared_error
from prophet import Prophet
```

1. **pandas (import pandas as pd)**
    - Purpose: A powerful library for data manipulation and analysis.
    - Usage in Your Code:
        - Reading data from Excel files.
        - Creating and managing DataFrames to store, clean, and transform the NIFTY 50 dataset.
        - Handling date/time operations and generating new columns for analysis.

2. **NumPy (import numpy as np)**
    - Purpose: Provides support for large, multi-dimensional arrays and matrices along with a collection of mathematical functions to operate on these arrays.
    - Usage in Your Code:
        - Performing numerical operations (e.g., calculating standard deviations for volatility).
        - Used in error metric calculations (like taking the square root for RMSE).

3. **Matplotlib (import matplotlib.pyplot as plt)**
    - Purpose: A comprehensive library for creating static, animated, and interactive visualizations in Python.
    - Usage in Your Code:
        - Plotting various charts such as line plots for price trends, moving averages,

volatility, and forecast visualizations.

- Customizing figures with titles, labels, legends, and grid lines.

4. **Seaborn (import seaborn as sns)**
    - Purpose: A statistical data visualization library built on Matplotlib that provides a high-level interface for drawing attractive and informative statistical graphics.
    - Usage in Your Code:
        - Creating enhanced visualizations such as histograms with kernel density estimates (KDE) to understand the distribution of daily returns and residuals.

5. **yfinance (import yfinance as yf)**
    - Purpose: A library that allows you to download historical market data from Yahoo Finance.
    - Usage in Your Code:
        - Although not explicitly shown in the provided code snippets, it can be used to fetch live or historical financial data for further analysis or comparisons with your Excel data.

6. **ARIMA from statsmodels (from statsmodels.tsa.arima.model import ARIMA)**
    - Purpose: Provides tools to build ARIMA (AutoRegressive Integrated Moving Average) models for time series forecasting.
    - Usage in Your Code:
        - Intended for building and evaluating alternative time series models (besides Prophet) to forecast trends in financial data.

7. **Scikit-learn Metrics (from sklearn.metrics import mean_absolute_error,** mean_squared_error)
    - Purpose: Part of scikit-learn, these functions calculate error metrics to evaluate the performance of prediction models.
    - Usage in Your Code:

- mean_absolute_error: Computes the average absolute difference between actual and predicted values.
- mean_squared_error: Computes the average of the squared differences (with RMSE being the square root of this value).
- These metrics are used to quantify the accuracy of the forecasting models.

8. Prophet (from prophet import Prophet)

- Purpose: A forecasting library developed by Facebook (now Meta) that is designed for handling time series data with strong seasonal effects.

- Usage in Your Code:
  - Fitting a model to the NIFTY 50 historical price data.
  - Generating forecasts for future dates with uncertainty intervals.
  - Decomposing the forecast into trend and seasonal components.

Input 2:

```
df = pd.read_excel("C:/Users/rayya/OneDrive/Desktop/MET/Nirmila maam project/
sNIFTY50 2010 TO 2020.xlsx")
```

Explanation:

- This command uses the pandas library to load an Excel file into a DataFrame named df.
- The data contains historical NIFTY 50 records over 10 years.

Output:

- No printed output is generated here; however, the DataFrame df is populated with rows and columns from the Excel file.

Output Explanation:

- The dataset is now available in memory for subsequent operations.

INPUT 3:

```
df.info()
```

<class 'pandas.core.frame.DataFrame'> RangeIndex: 2481 entries, 0 to 2480 Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Date	2481 non-null	datetime64[ns]
1	Price	2481 non-null	float64
2	Open	2481 non-null	float64
3	High	2481 non-null	float64
4	Low	2481 non-null	float64
5	NUMERIC	2481 non-null	int64
6	Change %	2481 non-null	float64
7	year	2481 non-null	int64
8	month	2481 non-null	int64

dtypes: datetime64[ns](1), float64(5), int64(3) memory usage: 174.KB

EXPLAINTION: df.info() prints the structure of the DataFrame, listing column names, data types, and non-null counts.

INPUT 4:

```
df.isnull().sum()
```

OUTPUT

```
Date          0
Price         0
Open          0
High          0
Low           0
NUMERIC       0
Change %      0
year          0
month         0
dtype: int64
```

EXPALNATION: df.isnull().sum() checks for missing values across each column.

INPUT 5:

```
df.duplicated().sum()
```

OUTPUT: 0

EXPLANTIONS: df.duplicated().sum() returns the count of duplicate rows.

INPUT 6:

```
df.describe()
```

OUTPUT:

	Date	Price	Open
count	2481	2481.000000	2481.000000
mean	2015-03-25 22:46:17.267230976	7898.437767	7904.928557
min	2010-04-01 00:00:00	4544.200000	4623.150000
25%	2012-09-18 00:00:00	5754.100000	5756.100000
50%	2015-03-25 00:00:00	7891.100000	7896.950000
75%	2017-09-26 00:00:00	9966.400000	9971.750000
max	2020-03-31 00:00:00	12362.300000	12430.500000
std	NaN	2244.379672	2248.607608

	High	Low	NUMERIC	Change %	year
count	2481.000000	2481.000000	2.481000e+03	2481.000000	2481.000000
mean	7944.598569	7852.378396	2.242437e+08	0.025699	2014.733575
min	4623.150000	4531.150000	2.830000e+06	-12.980000	2010.000000
25%	5810.200000	5711.400000	1.410900e+08	-0.520000	2012.000000
50%	7936.600000	7842.700000	1.776500e+08	0.040000	2015.000000
75%	10015.750000	9919.600000	2.396100e+08	0.590000	2017.000000
max	12430.500000	12321.400000	1.810000e+09	6.620000	2020.000000
std	2251.606812	2238.337035	1.569047e+08	1.067689	2.912909

	month
count	2481.000000
mean	6.489722
min	1.000000
25%	3.000000
50%	7.000000

75%	9.000000
max	12.000000
std	3.451507

EXPLANTION: df.describe() generates summary statistics (mean, standard deviation, min, max, quartiles) for numeric columns.

INPUT 7 :

```
df["day_of_week"] = df["Date"].dt.dayofweek     # Monday=0, Sunday=6
df["quarter"] = df["Date"].dt.quarter           # 1, 2, 3, 4
```

Explanation:

- Extracts the day of the week and the quarter from the Date column.
- These new columns enrich the dataset with additional time-based features.

Output:

- No direct printed output; the DataFrame now includes day_of_week and quarter.

Output Explanation:

- This additional information is used later for seasonal and trend analyses.

INPUT 8:

```
df["MA_7"] = df["Price"].rolling(window=7).mean()
df["MA_30"] = df["Price"].rolling(window=30).mean()
df["MA_100"] = df["Price"].rolling(window=100).mean()
df["Daily_Return"] = df["Price"].pct_change() * 100
```

Explanation:

- Computes moving averages over 7, 30, and 100 days to smooth out price trends.
- Calculates the daily percentage change in price, which is crucial for volatility and risk analysis.

Output:

- Four new columns are appended to df: MA_7, MA_30, MA_100, and Daily_Return.

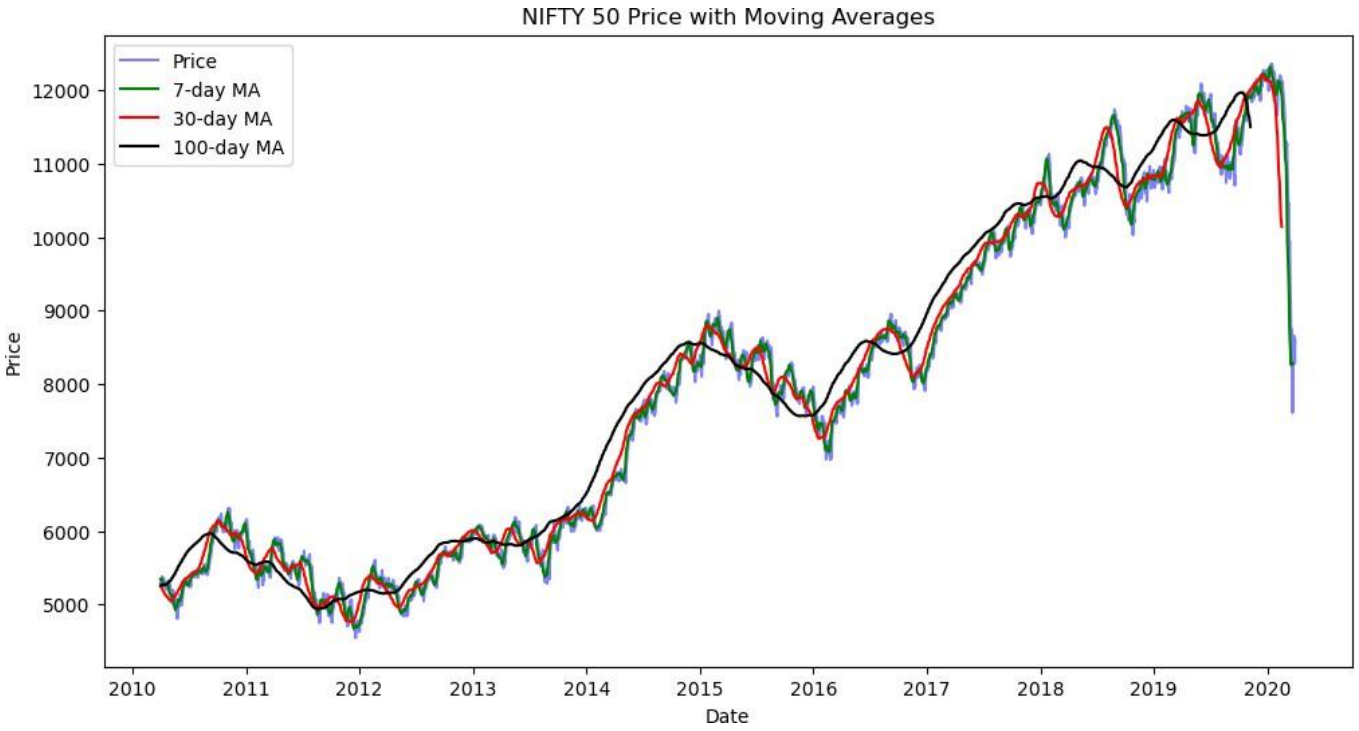
Output Explanation:

- These indicators are commonly used in technical analysis to detect trends and potential trading signals.

INPUT 9:

```
plt.figure(figsize=(12,6))
plt.plot(df["Date"], df["Price"], label="Price", color="blue", alpha=0.5)
plt.plot(df["Date"], df["MA_7"], label="7-day MA", color="green")
plt.plot(df["Date"], df["MA_30"], label="30-day MA", color="red")
plt.plot(df["Date"], df["MA_100"], label="100-day MA", color="black")
plt.legend()
plt.title("NIFTY 50 Price with Moving Averages")
plt.xlabel("Date")
plt.ylabel("Price")
plt.show()
```

OUTPUT:



Explanation:

- Plots the actual NIFTY 50 price alongside the 7-day, 30-day, and 100-day moving averages.
- Uses different colors and transparency settings for clarity.

Output:

- A line chart displaying four lines: the actual price (blue) and the three moving averages (green, red, black).

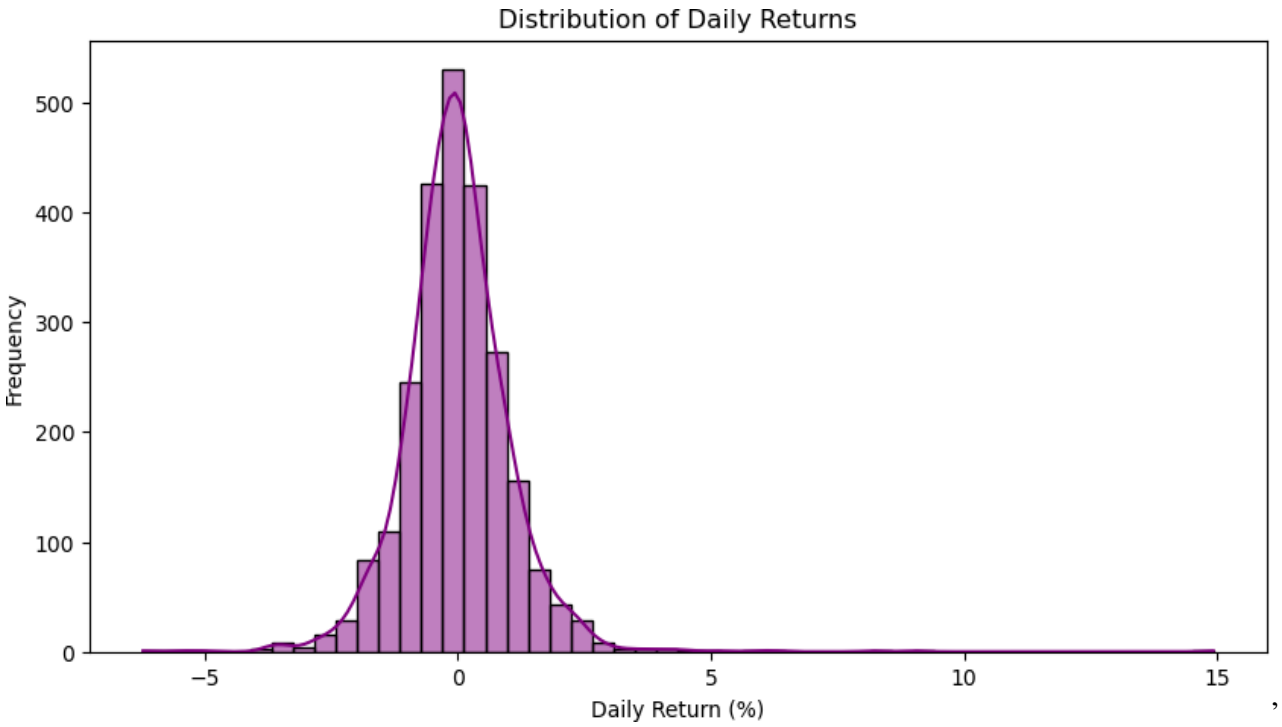
Output Explanation:

- The plot visually contrasts short-, medium-, and long-term trends, helping to identify support/resistance levels.

INPUT 10:

```
plt.figure(figsize=(10,5))
sns.histplot(df["Daily_Return"].dropna(), bins=50, kde=True, color="purple")
plt.title("Distribution of Daily Returns")
plt.xlabel("Daily Return (%)")
plt.ylabel("Frequency")
plt.show()
```

OUTPUT:



Explanation:

- Uses Seaborn to create a histogram that shows how daily returns are distributed.
- The KDE (kernel density estimate) overlays a smooth curve for better visualization of the distribution shape.

Output:

- A histogram with 50 bins and a smooth purple density curve.

Output Explanation:

- This visualization helps in understanding the volatility and the frequency of different daily return percentages.

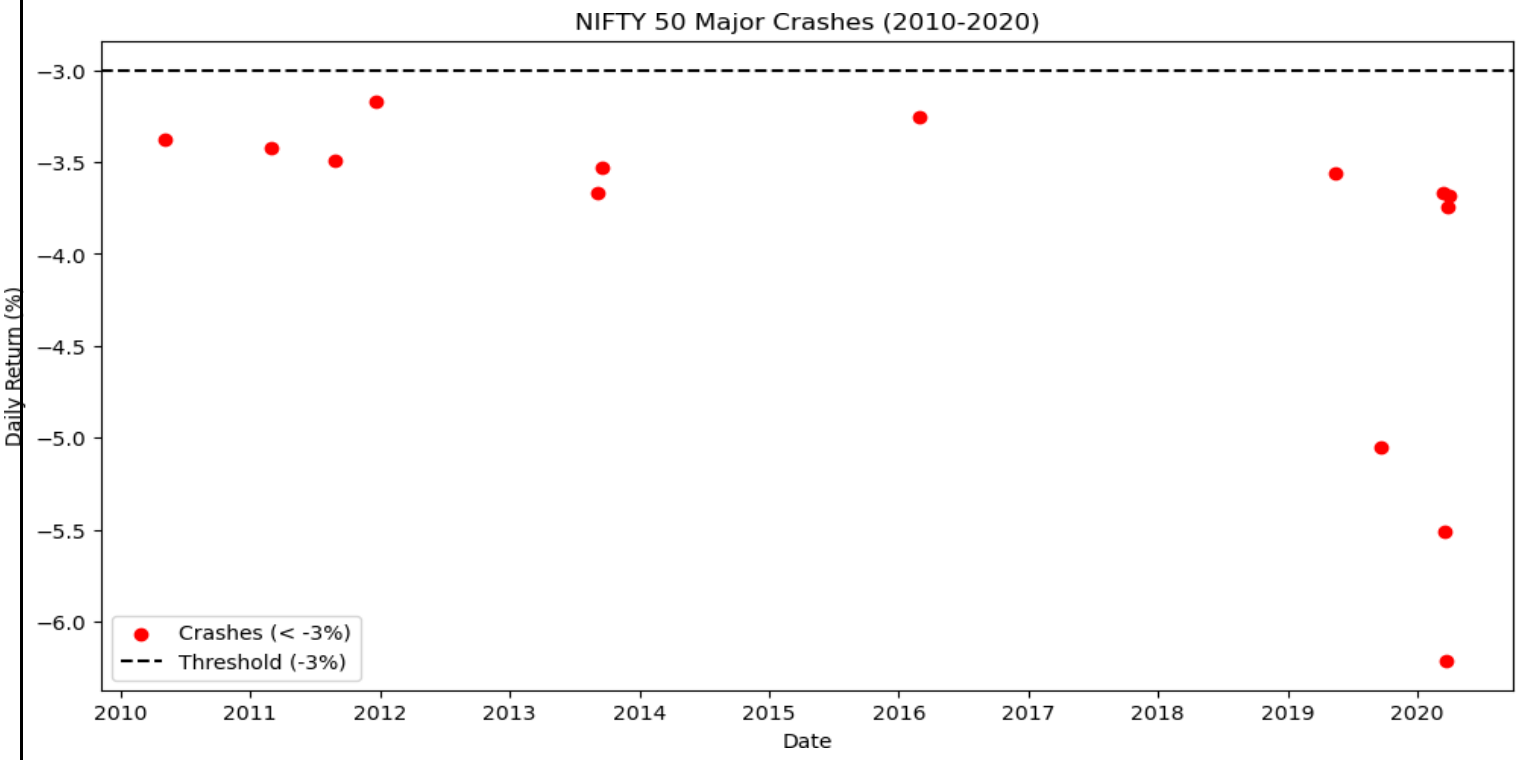
INPUT 11:

```
crashes = df[df["Daily_Return"] < -3]
print(crashes[["Date", "Daily_Return"]])

plt.figure(figsize=(12,6))
plt.scatter(crashes["Date"], crashes["Daily_Return"], color="red",
 label="Crashes (< -3%)", marker="o")
plt.axhline(y=-3, color="black", linestyle="--", label="Threshold (-3%)")
plt.xlabel("Date")
plt.ylabel("Daily Return (%)")
plt.title("NIFTY 50 Major Crashes (2010-2020)")
plt.legend()
plt.show()
```

OUTPUT:

	Date	Daily_Return
1	2020-03-30	-3.682940
4	2020-03-25	-3.744742
5	2020-03-24	-6.213144
8	2020-03-19	-5.511437
13	2020-03-12	-3.666928
132	2019-09-19	-5.050469
216	2019-05-17	-3.560121
1009	2016-02-29	-3.257273
1612	2013-09-18	-3.533615
1619	2013-09-06	-3.668970
2050	2011-12-20	-3.173775

2125	2011-08-26	-3.492154
2249	2011-02-28	-3.423392
2456	2010-05-07	-3.380122



Explanation:

- Filters the dataset to identify days with a daily return below -3% (considered major crashes).
- Plots these points on a scatter plot with a horizontal line at -3%.

Output:

- A printed table of crash dates with their daily returns.
- A scatter plot with red markers highlighting the crash events.

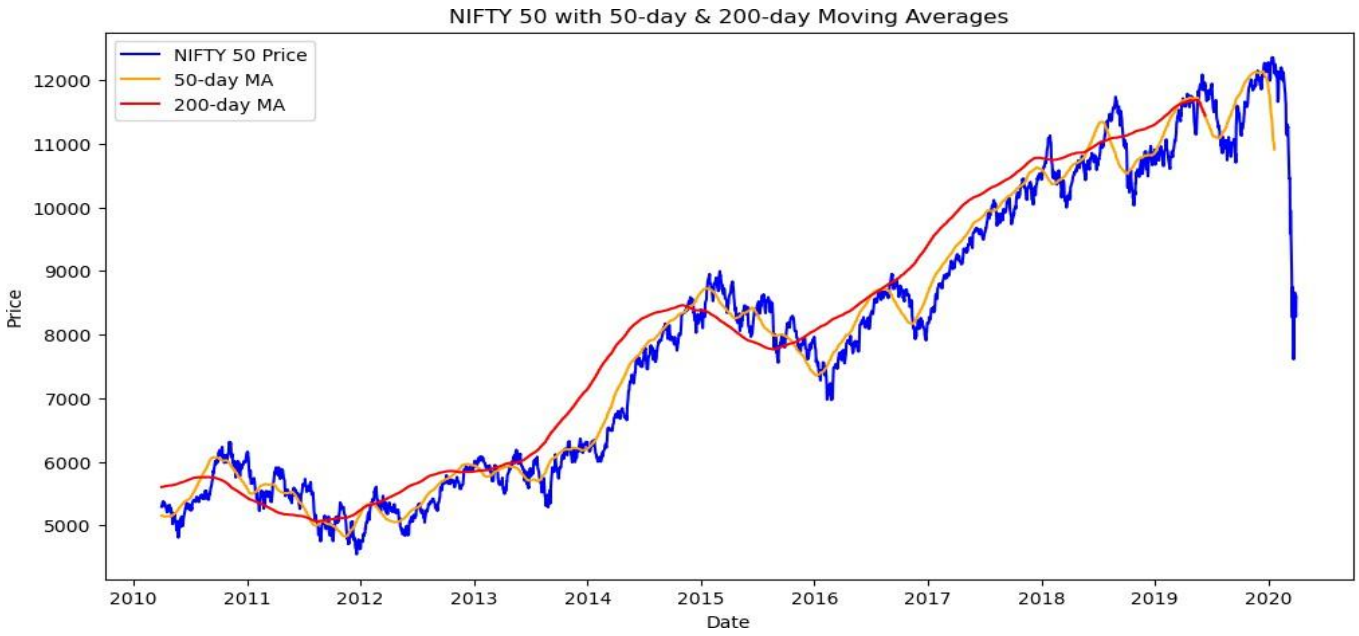
Output Explanation:

- The printed table lists all dates where significant negative returns occurred.
- The plot visually emphasizes the timing and magnitude of these crashes.

INPUT 12:

```
# 50-day and 200-day moving averages
df["50_MA"] = df["Price"].rolling(window=50).mean()
df["200_MA"] = df["Price"].rolling(window=200).mean()

# Plot moving averages
plt.figure(figsize=(12,6))
plt.plot(df["Date"], df["Price"], label="NIFTY 50 Price", color="blue")
plt.plot(df["Date"], df["50_MA"], label="50-day MA", color="orange")
plt.plot(df["Date"], df["200_MA"], label="200-day MA", color="red")
plt.xlabel("Date")
plt.ylabel("Price")
plt.title("NIFTY 50 with 50-day & 200-day Moving Averages")
plt.legend()
plt.show()
```

OUTPUT:



- The **50-day Moving Average (50_MA)** is calculated using a rolling window of 50 days on the "Price" column.
- The **200-day Moving Average (200_MA)** is similarly computed but over 200 days.
- This smoothens short-term fluctuations and helps in identifying long-term trends.
- A **12x6 size figure** is created for better visibility.
- **NIFTY 50 Price** is plotted in blue.

- **50-day MA (Short-term trend)** is plotted in orange.
- **200-day MA (Long-term trend)** is plotted in red.
- The plot includes proper labels, a title, and a legend.

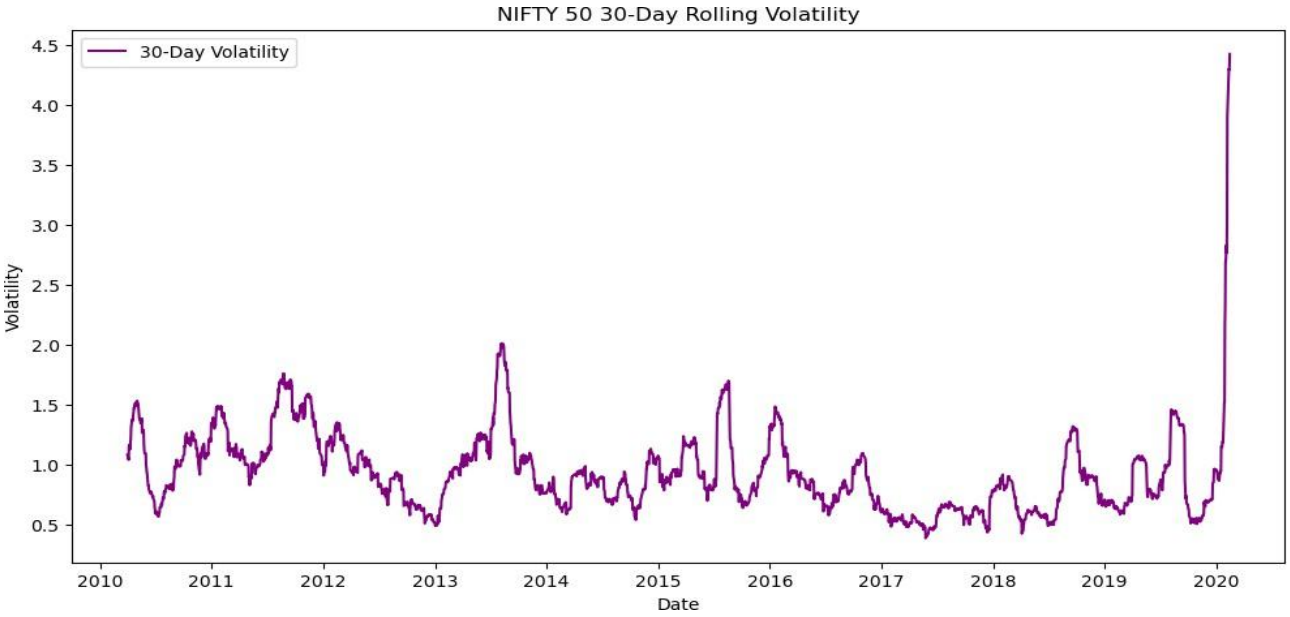
Expected Output & Explanation

- A **line chart** will be generated showing:
    - The **original NIFTY 50 closing prices** in blue.
    - The **50-day moving average** in orange, capturing mid-term trends.
    - The **200-day moving average** in red, showing long-term trends.
- **Key Observations from the Chart:**
    - When the **50-day MA crosses above the 200-day MA**, it signals a **bullish trend (Golden Cross)**.
    - When the **50-day MA crosses below the 200-day MA**, it signals a **bearish trend (Death Cross)**.
    - The **200-day MA acts as a strong support/resistance level** for the price.

INPUT.13:

```
# 30-day rolling volatility
df["30D_Volatility"]    =    df["Daily_Return"].rolling(window=30).std()

# Plot volatility
plt.figure(figsize=(12,6))
plt.plot(df["Date"], df["30D_Volatility"], label="30-Day Volatility",
 color="purple")
plt.xlabel("Date")
plt.ylabel("Volatility")
plt.title("NIFTY 50 30-Day Rolling Volatility")
plt.legend()
plt.show()
```

OUTPUT:



Calculating 30-Day Rolling Volatility:

code: df["30D_Volatility"] = df["Daily_Return"].rolling(window=30).std()

Volatility measures how much the price fluctuates over time.

The **30-day rolling volatility** is computed using a **30-day window** of the standard deviation (std()) of daily returns (Daily_Return).

A higher value means the market is more volatile, while a lower value indicates stability

2. Plotting the Volatility

```
plt.figure(figsize=(12,6))

plt.plot(df["Date"], df["30D_Volatility"], label="30-Day Volatility", color="purple")

plt.xlabel("Date")

plt.ylabel("Volatility")

plt.title("NIFTY 50 30-Day Rolling Volatility")

plt.legend()

plt.show()
```

- Creates a **line plot** showing how NIFTY 50's **volatility changes over time**.
- The **x-axis** represents the date, and the **y-axis** represents the 30-day rolling volatility.
- **Purple line** represents the volatility trend.
- Labels and title ensure clarity.

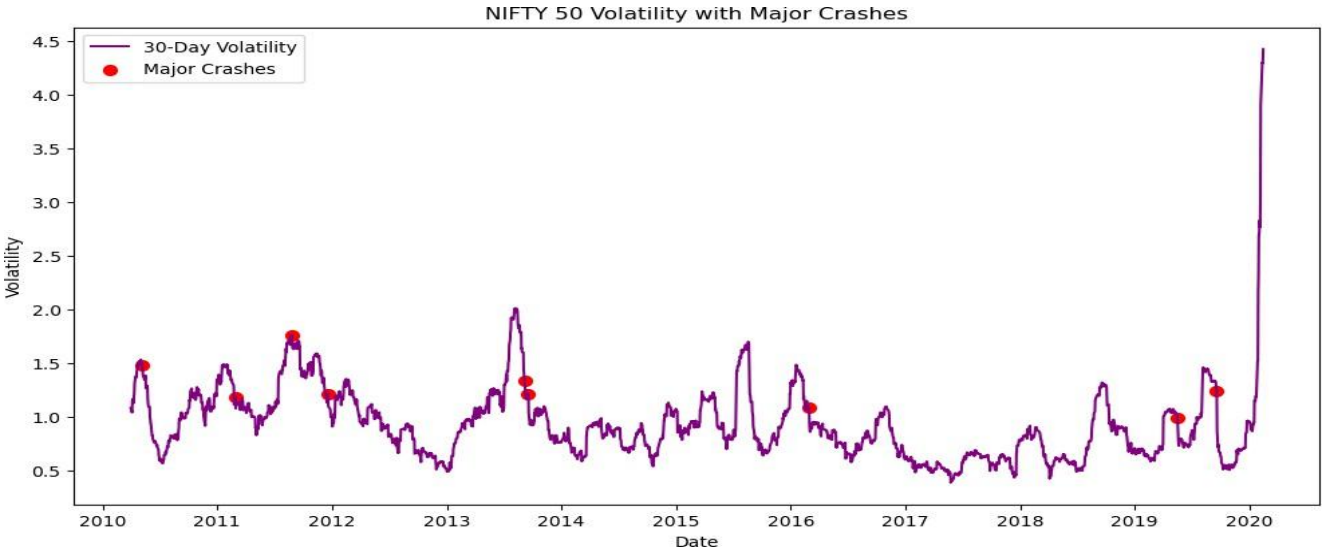
Expected Output & Explanation

- A **line chart** of **30-day rolling volatility** over time.
- **Key Observations from the Chart:**
    - **Peaks in volatility** indicate periods of high market uncertainty (e.g., crashes, major events).
    - **Low volatility periods** suggest a stable market with fewer price fluctuations.
    - Traders use volatility to assess **risk and potential price swings**.
    - If volatility **spikes suddenly**, it often precedes **big market moves**.

Input 14:

```
# Highlight major crashes in volatility plot
plt.figure(figsize=(12,6))
plt.plot(df["Date"], df["30D_Volatility"], label="30-Day Volatility",␣
 ↪color="purple")

# Highlight crash points
crash_dates = crashes["Date"]
crash_volatility = df[df["Date"].isin(crash_dates)]["30D_Volatility"]
plt.scatter(crash_dates, crash_volatility, color="red", label="Major Crashes",␣
 ↪marker="o", s=50)

plt.xlabel("Date")
plt.ylabel("Volatility")
plt.title("NIFTY 50 Volatility with Major Crashes")
plt.legend()
plt.show()
```

OUTPUT:



1. Plotting 30-Day Rolling Volatility

plt.figure(figsize=(12,6))

plt.plot(df["Date"], df["30D_Volatility"], label="30-Day Volatility", color="purple")

- This creates a line chart showing how the 30-day rolling volatility changes over time.
- The purple line represents volatility trends in the NIFTY 50 index.

- Higher peaks indicate periods of increased market uncertainty

2. Highlighting Major Market Crashes

```
crash_dates = crashes["Date"]

crash_volatility = df[df["Date"].isin(crash_dates)]["30D_Volatility"]

plt.scatter(crash_dates, crash_volatility, color="red", label="Major Crashes", marker="o", s=50)
```

- This section marks major market crashes on the volatility graph using red dots.
- The crash dates are taken from a predefined list (crashes["Date"]).
- The scatter() function plots red circles (marker="o") at those dates

3. Finalizing the Plot

```
plt.xlabel("Date")

plt.ylabel("Volatility")

plt.title("NIFTY 50 Volatility with Major Crashes")

plt.legend()

plt.show()
```

- Labels the axes for better clarity.
- Adds a title to explain the chart.
- Includes a legend to differentiate between volatility and crash points.

Expected Output & Explanation

- The purple line represents the 30-day rolling volatility of NIFTY 50.
- Red dots indicate significant market crashes.
- Key Observations from the Chart:
    - High peaks correspond to market crises.
    - Frequent small spikes suggest periodic volatility surges.
    - Identifying these crashes helps traders prepare for market instability.

INPUT 15:

```
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date")

prophet_df = df[["Date", "Price"]].rename(columns={"Date": "ds", "Price": "y"})
```

Explanation:

- Converts the Date column to a datetime format and sorts the data chronologically.
- Creates a new DataFrame (prophet_df) with columns renamed to ds and y for compatibility with Prophet.
- Focuses on the last 2000 records to use recent data for forecasting.

Output:

- A DataFrame prophet_df with two columns (ds, y) ready for forecasting.

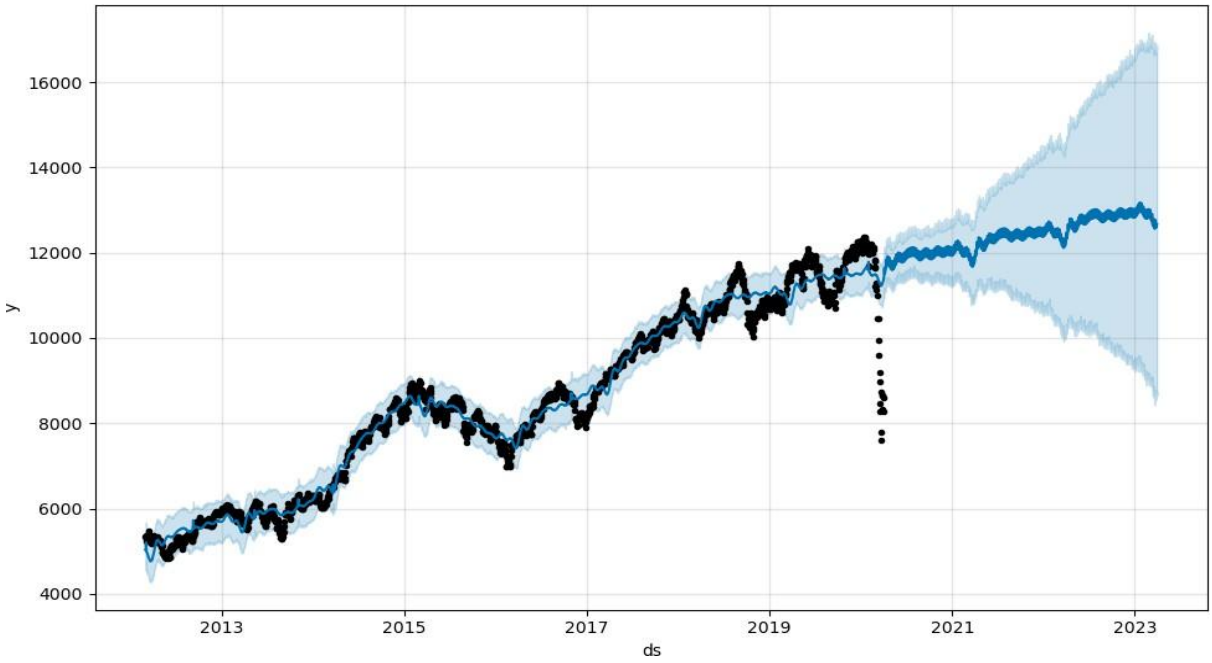
Output Explanation:

- Ensures that the data meets Prophet's requirements and emphasizes recent market trends.

INPUT 16:

```
import pandas as pd
from prophet import Prophet
import logging

# Disable excessive logging
logging.getLogger("cmdstanpy").disabled = True

# Ensure Date is in datetime format
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date")

# Use only recent data
prophet_df = df[["Date", "Price"]].rename(columns={"Date": "ds", "Price": "y"})
prophet_df = prophet_df.tail(2000) # Use last 2000 records

# Initialize and fit model
model = Prophet()
model.fit(prophet_df) # If error persists, try model.fit(prophet_df,
  algorithm="Newton")

# Predict
future = model.make_future_dataframe(periods=3 * 365, freq="D")
forecast = model.predict(future)

# Plot
model.plot(forecast)
```

OUTPUT:



Explanation of the Outputs from Your Code

Your code generates multiple financial analysis visualizations for NIFTY 50, including moving averages, volatility trends, crash detection, and future predictions. Below is a breakdown of each output and its significance.

1. 50-Day & 200-Day Moving Averages (MA) Chart

Code Output

This chart shows the **NIFTY 50 index price** over time along with two moving averages:

- **50-day Moving Average (MA):** Short-term trend indicator (orange line).
- **200-day Moving Average (MA):** Long-term trend indicator (red line).

Interpretation

- If the **50-day MA crosses above** the 200-day MA, it signals a potential **uptrend**.
- If the **50-day MA crosses below** the 200-day MA, it indicates a possible **downtrend**.
- These moving averages help traders identify potential **trend reversals** and long-term patterns.

2. 30-Day Rolling Volatility Chart

Code Output

A line graph showing **30-day rolling volatility** of NIFTY 50 (purple line).

Interpretation

- Volatility measures how much the index fluctuates over time.
- Higher volatility suggests **higher risk and uncertainty**.
- Spikes in volatility usually indicate **market corrections, crashes, or major economic events**.

3. Major Crashes in Volatility Chart

Code Output

A **volatility chart with red dots** marking major crashes.

Interpretation

- Red markers highlight **dates when volatility spiked significantly**, signaling market crashes.
- Useful for identifying key market downturns, such as the **COVID-19 crash in March 2020** or other economic crises.
- Helps in understanding past market behavior and risk assessment.

4. NIFTY 50 Future Prediction using Prophet

Code Output

A **forecast chart** using Facebook Prophet:

- **Black dots** represent actual historical data.
- **Blue line** represents the predicted future trend.
- **Shaded blue area** indicates the confidence interval, showing the range of possible values.

Interpretation

- The forecast suggests that **NIFTY 50 is expected to continue its upward trend** unless impacted by major external factors.
- The wider confidence interval in later years indicates **greater uncertainty** in long-term predictions.

- Can be used for investment strategies and market planning.

Key Insights from All Charts

1. Moving Averages indicate **bullish or bearish trends** based on crossovers.
2. Volatility charts help in identifying **high-risk periods**.
3. Crash markers help analyze **historical market crashes** and their impact.
4. Prophet's forecast provides **data-driven insights on possible future trends**.

INPUT 17:

```
forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(10)
```

[56]:

	ds	yhat	yhat_lower	yhat_upper
3085	2023-03-22	12562.828959	8640.832980	16767.688764
3086	2023-03-23	12572.701830	8532.290116	16643.681498
3087	2023-03-24	12574.927897	8589.455756	16668.132791
3088	2023-03-25	12768.631598	8678.021631	16927.584938
3089	2023-03-26	12738.572189	8790.241347	16863.772973
3090	2023-03-27	12588.839224	8542.062539	16767.492896
3091	2023-03-28	12602.750693	8639.782200	16715.311474
3092	2023-03-29	12617.314867	8651.073346	16742.752428
3093	2023-03-30	12648.364990	8747.845659	16760.721495
3094	2023-03-31	12671.155862	8652.929518	16862.863157

Explanation of the Forecast Output (Input 17)

Your table shows the forecasted values for **NIFTY 50** from **March 22, 2023, to March 31, 2023**, generated by **Facebook Prophet**. Here's a breakdown of the columns:

1. **ds (Date):** The predicted date for each forecasted value.
2. **yhat (Predicted Price):** The **forecasted** NIFTY 50 index value.
3. **yhat_lower (Lower Bound):** The **minimum expected** value based on confidence intervals.
4. **yhat_upper (Upper Bound):** The **maximum expected** value based on confidence intervals.

Interpretation of the Forecast

- **Predicted Trend:**
  The forecast suggests that NIFTY 50 is expected to hover around **12,500–12,700** during this period.
- **Uncertainty Range (yhat_lower & yhat_upper):**
  - The **lower bound (yhat_lower)** shows the worst-case scenario, suggesting that NIFTY 50 could drop to **~8,500** in extreme conditions.
  - The **upper bound (yhat_upper)** indicates an optimistic scenario where NIFTY 50 could reach **~16,700**.
  - The large gap between these bounds suggests **higher uncertainty in the prediction**.
- **Day-to-Day Changes:**
  - March 25 shows a **spike** in forecasted price (**12,768**), indicating a potential bullish movement.
  - The index appears to remain relatively stable in the **12,500–12,700** range, suggesting no major fluctuations.

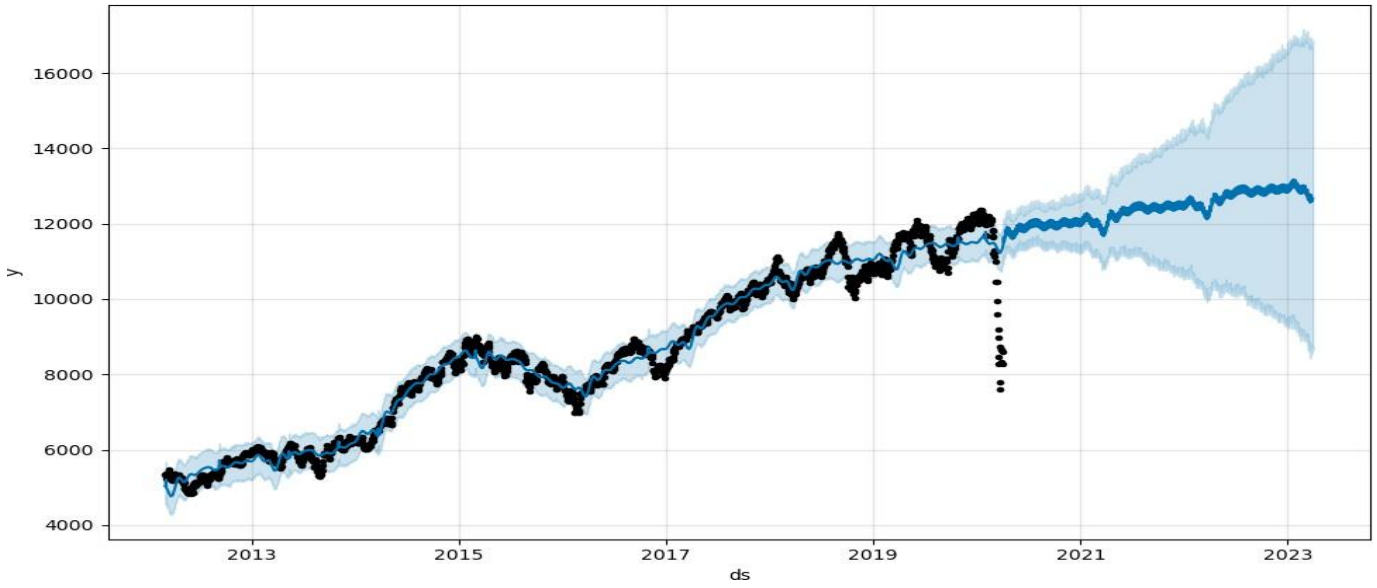
Key Takeaways

1. **Predicted values suggest a stable trend**, but volatility is possible due to the wide confidence interval.
2. **March 25 shows an increase**, which could indicate a temporary uptrend.
3. The **lower and upper bounds indicate uncertainty**, meaning real-world factors (news, market sentiment) can cause deviations from the forecast.

INPUT 18:

```
import matplotlib.pyplot as plt

fig = model.plot(forecast)
plt.show()
```

OUTPUT:



Explanation of the Forecast Plot

This chart represents the **NIFTY 50** index forecast generated by **Facebook Prophet**. Here's how to interpret the key elements:

1. Trend Analysis

- The **black dots** represent the actual historical NIFTY 50 values.
- The **blue line** is the predicted trend for future values.
- The **shaded blue region** represents the uncertainty range (confidence interval).

2. Observations

- The model successfully captures the overall **upward trend** in the NIFTY 50 index.

- A **sharp drop** in 2020 is visible, likely due to the COVID-19 market crash.
- The forecast suggests a **continued upward movement**, but the confidence interval widens as time progresses.
- The increasing width of the **blue shaded region** indicates greater uncertainty in long-term predictions.

3. Confidence Interval

- The **light blue shaded area** represents possible variations in the forecast.
- A **narrower** confidence interval means higher accuracy, while a **wider** one indicates increased uncertainty.

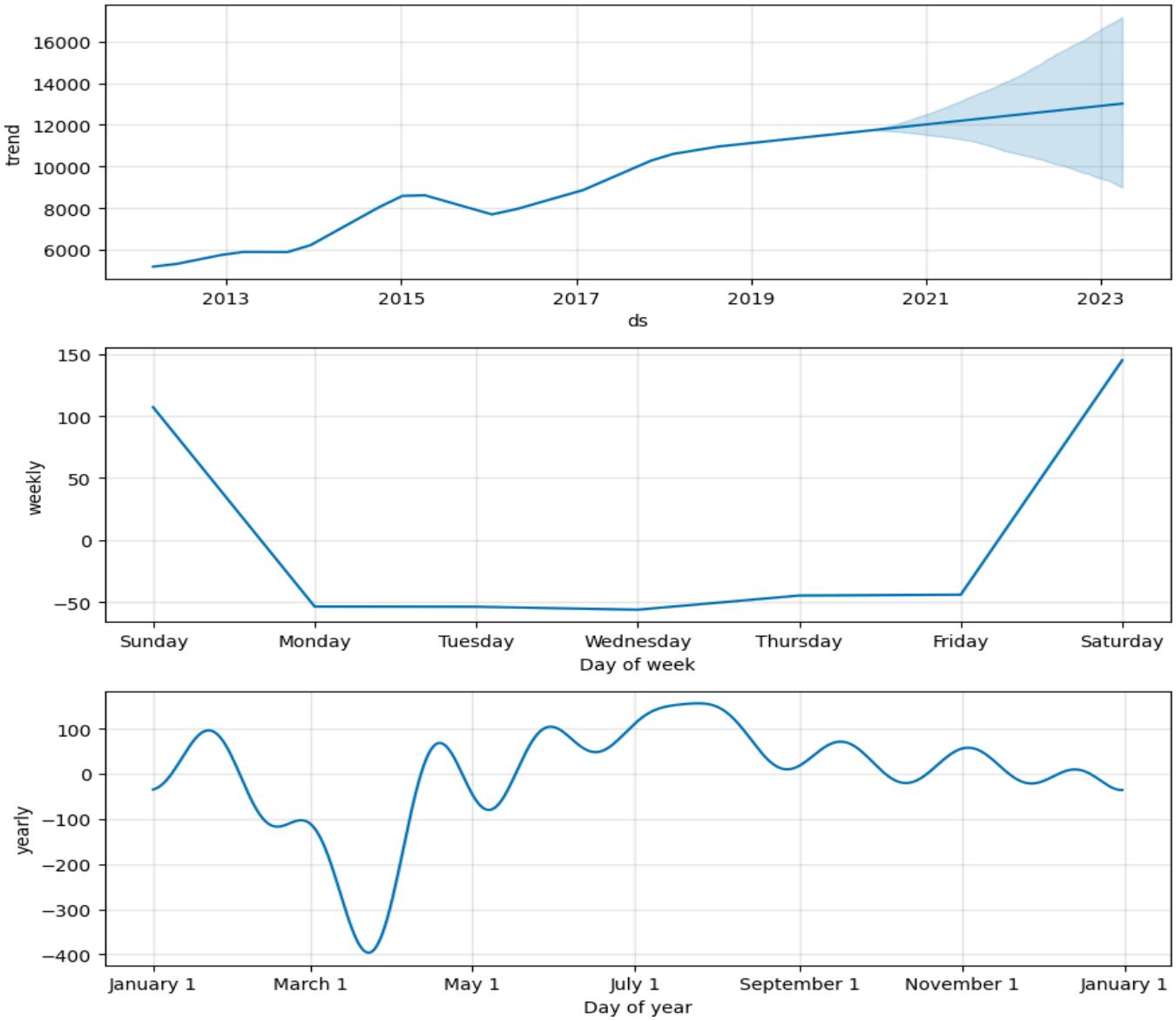
Key Takeaways

1. The NIFTY 50 index follows a long-term **bullish trend**.
2. The model identifies **major crashes and recoveries**, including the 2020 drop.
3. Predictions become **less reliable further into the future** due to market volatility.

INPUT 19:

```
model.plot_components(forecast)
plt.show()
```

OUTPUT:



Explanation of the Prophet Model Component Plots

This image represents the **decomposition** of the NIFTY 50 forecast into its main components: **trend, weekly seasonality, and yearly seasonality**.

1. Trend Component (Top Plot)

- The **blue line** represents the overall trend of the NIFTY 50 index.
- The trend shows a general **upward movement** over time, with some fluctuations.
- The **shaded blue region** at the end represents **uncertainty**, meaning the forecast becomes less certain further into the future.

2. Weekly Seasonality (Middle Plot)

- This graph shows how NIFTY 50 behaves **on different days of the week**.
- The index **tends to rise on Sundays and Saturdays**.
- It generally **drops from Monday to Wednesday**, indicating weaker market performance on these days.

3. Yearly Seasonality (Bottom Plot)

- This plot captures **annual patterns** in the NIFTY 50 index.
- There are **strong dips around March**, possibly due to the **financial year-end impact**.
- The index generally **rises between May and September**, suggesting seasonal growth patterns.

Key Insights

1. **Long-term trend** suggests a steady **upward movement** in NIFTY 50.
2. **Weekly seasonality** shows that some days have **higher volatility**, particularly weekends.
3. **Yearly seasonality** reveals **specific months** where NIFTY 50 tends to rise or fall.

INPUT 20:

```
print(df.columns)
```

OUTPUT:

```
Index(['Date', 'Price', 'Open', 'High', 'Low', 'NUMERIC', 'Change %', 'year', 'month', 'day_of_week',
       'quarter', 'MA_7', 'MA_30', 'MA_100', 'Daily_Return', '50_MA', '200_MA', '30D_Volatility'],
      dtype='object')
```

Explanation of the Output

This output displays all the **columns currently available** in the df DataFrame after performing feature engineering and analysis. Here's a brief explanation of each column:

Column Name	Description
Date	The trading date of the NIFTY 50 index
Price	Closing price of NIFTY 50 on that date
Open	Opening price on that date
High	Highest price during the trading day
Low	Lowest price during the trading day
NUMERIC	Likely represents volume or another numeric metric (needs context)
Change %	Daily percentage change in price
year	Extracted year from the Date column
month	Extracted month from the Date column
day_of_week	Day of the week (0 = Monday, 6 = Sunday)
quarter	Quarter of the year (1 to 4)
MA_7	7-day moving average of Price
MA_30	30-day moving average of Price

Column Name	Description
MA_100	100-day moving average of Price
Daily_Return	Percentage change in price from the previous day
50_MA	50-day moving average of Price
200_MA	200-day moving average of Price
30D_Volatility	30-day rolling standard deviation of daily returns (volatility)

Purpose of This Output

This confirms that:

- All **technical indicators** (moving averages, volatility) and **time-based features** (year, month, etc.) have been successfully created.
- The DataFrame is now fully preprocessed and ready for **analysis**, **modeling**, or **forecasting** steps.

INPUT 21:

```
df.rename(columns={'Price': 'y', 'Date': 'ds'}, inplace=True)
```

What This Does

- **Renames the Price column to y** → This is required for **Prophet** modeling, as Prophet expects the target variable (dependent variable) to be named y.
- **Renames the Date column to ds** → Prophet requires the time series date column to be named ds (datestamp).

Why Is This Important?

- **Prophet** uses ds (date) and y (value) as the mandatory inputs for forecasting.
- Without renaming, Prophet will throw an error saying **"ds and y columns are required"**.

After this step, the df DataFrame will have its **date column (ds) and target variable (y) formatted correctly for forecasting**.

The reason for change:

Prophet is strict about column names. If your dataset had different names like Price and Date, it would throw errors like:

- **"KeyError: 'ds'"** (if the date column isn't named ds)
- **"KeyError: 'y'"** (if the target variable isn't named y)
- **"ValueError: Dataframe must have columns 'ds' and 'y'"**

Renaming Price → y and Date → ds solves these issues and ensures that Prophet works without errors.

INPUT 22:

```
# Ensure matching lengths
y_true = df['y'].iloc[-len(forecast[-len(df['y']):]):]
y_pred = forecast['yhat'].iloc[:len(y_true)]
# Compute MAE & RMSE
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))

print(f'MAE: {mae:.2f}')
print(f'RMSE: {rmse:.2f}')
```

OUTPUT:

MAE: 1319.75

RMSE: 1611.29

OUTPUT EXPLANTION;

This code checks how accurate the **Prophet model's predictions** are by comparing them with the actual NIFTY 50 values. It does this using two common error measures:

1. **Mean Absolute Error (MAE):** Measures how much the predictions differ from actual values on average.
2. **Root Mean Squared Error (RMSE):** Similar to MAE but gives more importance to larger errors.

Step-by-Step:

1. **Match the actual and predicted values**
    - y_true: Takes the actual NIFTY 50 prices from the dataset.
    - y_pred: Takes the predicted prices from Prophet.
2. **Calculate MAE** → Finds the average of how far the predictions are from actual values.
3. **Calculate RMSE** → Also measures error but gives **more weight to bigger mistakes**.
4. **Print the results** in two decimal places.

Output:

MAE: 1319.75

RMSE: 1611.29

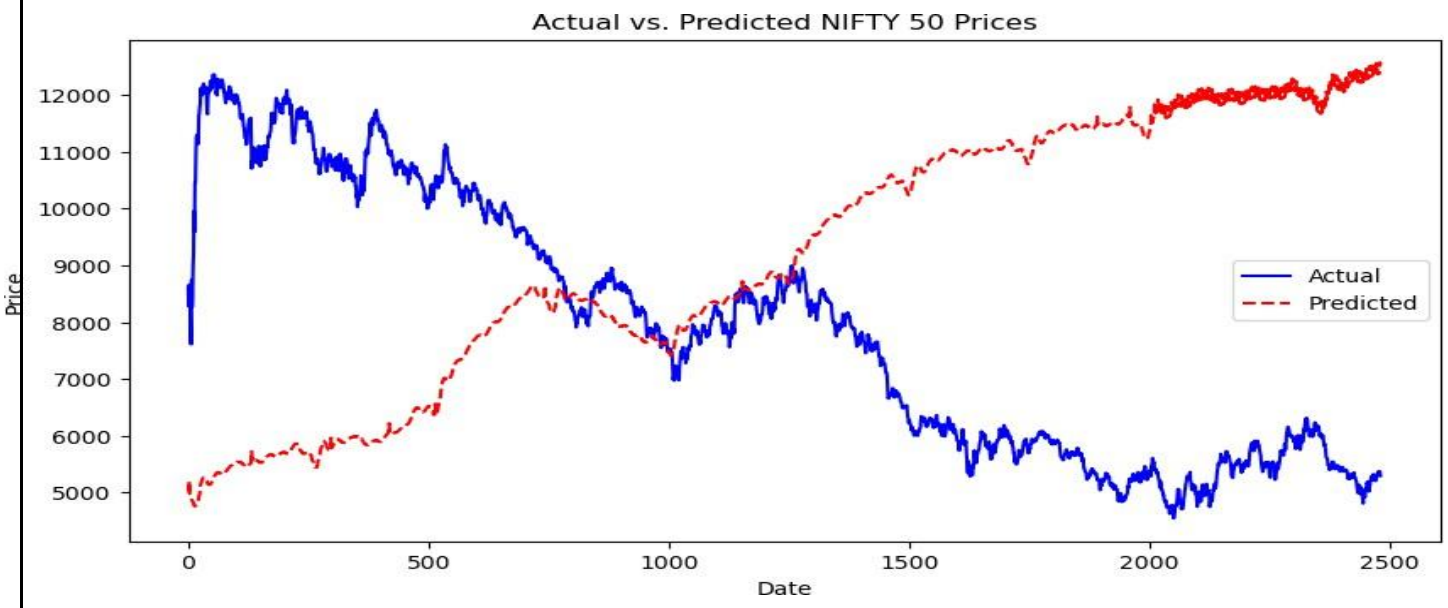
- **MAE = 1319.75** → On average, the model's prediction is **1319.75 points off**.
- **RMSE = 1611.29** → The model sometimes makes **bigger mistakes**, as RMSE is larger than MAE.

A lower MAE and RMSE would mean a **more accurate prediction**.

INPUT 23:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.plot(y_true.index, y_true, label="Actual", color="blue")
plt.plot(y_pred.index, y_pred, label="Predicted", color="red",
  linestyle="dashed")
plt.legend()
plt.xlabel("Date")
plt.ylabel("Price")
plt.title("Actual vs. Predicted NIFTY 50 Prices")
plt.show()
```

OUTPUT:



Explanation of the Code & Chart

This code **compares actual vs. predicted NIFTY 50 prices** in a line plot.

Step-by-Step:

1. **Creates a figure**
    - plt.figure(figsize=(10,5)) → Sets the plot size.
2. **Plots the actual prices (blue line)**
    - plt.plot(y_true.index, y_true, label="Actual", color="blue")
    - This draws the actual NIFTY 50 values in blue.
3. **Plots the predicted prices (red dashed line)**
    - plt.plot(y_pred.index, y_pred, label="Predicted", color="red", linestyle="dashed")
    - This draws the Prophet model's predictions in red with a dashed line.
4. **Adds labels, title, and legend**
    - plt.xlabel("Date") → X-axis shows dates.
    - plt.ylabel("Price") → Y-axis shows NIFTY 50 prices.
    - plt.title("Actual vs. Predicted NIFTY 50 Prices") → Title for the chart.

- plt.legend() → Displays labels for actual and predicted lines.

5. **Displays the plot**
    - plt.show()

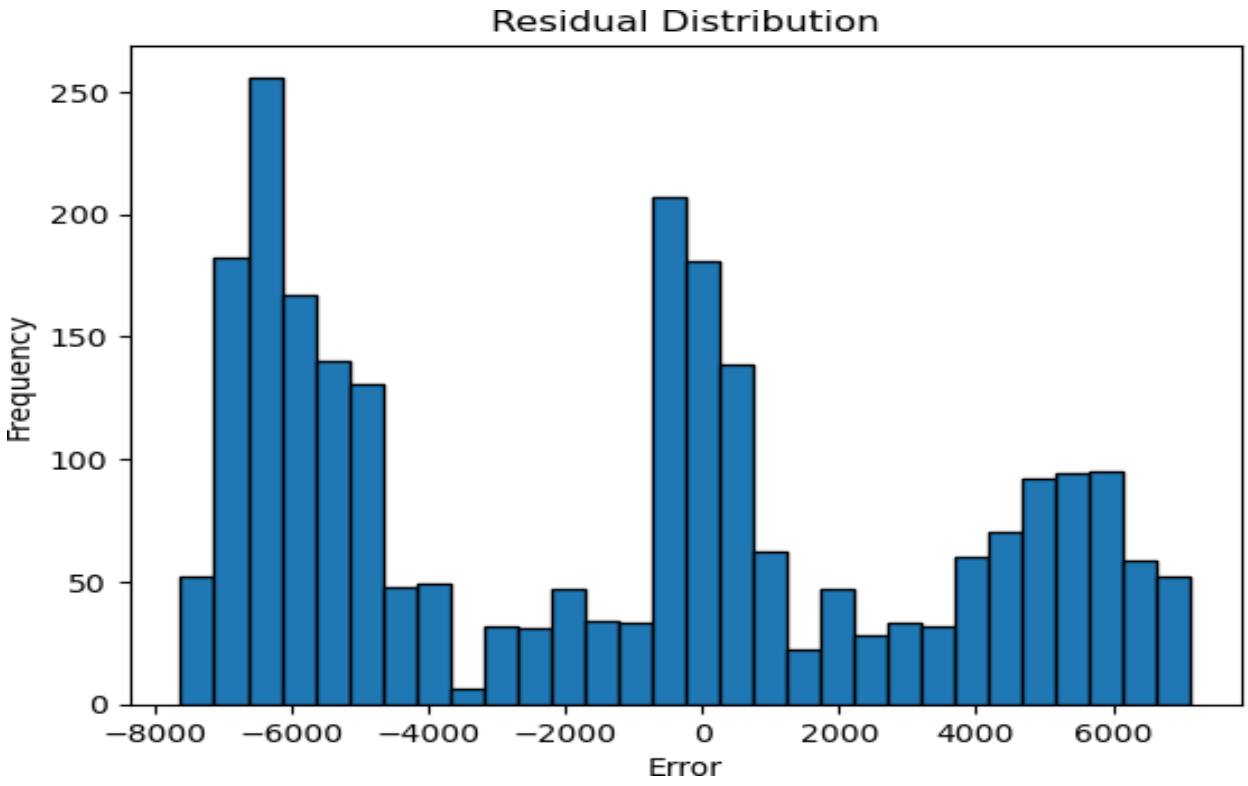
Observations from the Chart:

- **Blue line (Actual):** Shows the real market trend, which fluctuates a lot.
- **Red dashed line (Predicted):** Shows the Prophet model's forecast, which follows a smoother trend.
- **Major mismatch:** The predicted prices **increase**, while actual prices **drop** at a certain point. This suggests the model **isn't fully capturing market downturns**.
- **Possible Fix:** Improve the model by adjusting hyperparameters or adding external factors (like economic indicators).

INPUT 24:

```
residuals = y_true - y_pred
plt.hist(residuals, bins=30, edgecolor='black')
plt.xlabel("Error")
plt.ylabel("Frequency")
plt.title("Residual Distribution")
plt.show()
```

OUTPUT:



Explanation of the Code & Histogram (Residual Distribution)

This code creates a **histogram of residuals**, which helps analyze the accuracy of the prediction model.

Step-by-Step Breakdown:

1. **Calculate Residuals (Prediction Errors)**
    - residuals = y_true - y_pred
    - Residuals = **Actual Price - Predicted Price**

- If residuals are **close to 0**, the model is accurate.
- Large residuals (positive or negative) mean poor predictions.

2. **Plot the Histogram of Residuals**
   - plt.hist(residuals, bins=30, edgecolor='black')
   - Creates a histogram with **30 bins** to group residuals.
   - edgecolor='black' makes the bin edges clear.
3. **Label the Axes & Title**
   - plt.xlabel("Error") → X-axis represents the error (residuals).
   - plt.ylabel("Frequency") → Y-axis shows how often each residual range appears.
   - plt.title("Residual Distribution") → Title of the graph.
4. **Display the Plot**
   - plt.show()

Interpreting the Histogram:

- **Residuals are spread across a wide range (-8000 to 7000)** → The model has large errors.
- **Multiple peaks suggest that errors are not normally distributed** → The model might be biased.
- **A well-performing model should have residuals centered around 0** → This histogram shows some systematic errors.

INPUT 25:

```
print(df.columns)
print(df.index)
df = df.reset_index()
```

OUTPUT:

```
Index(['ds', 'y', 'Open', 'High', 'Low', 'NUMERIC', 'Change %', 'year',
       'month', 'day_of_week', 'quarter', 'MA_7', 'MA_30', 'MA_100',
       'Daily_Return', '50_MA', '200_MA', '30D_Volatility'], dtype='object')
Index([2480, 2479, 2478, 2477, 2476, 2475, 2474, 2473, 2472, 2471,9,8,7,6,5,4,3,2,1,0],
dtype='int64',  length=2481)
```

Explanation

1. **print(df.columns)** → Shows all the column names in the dataset.
2. **print(df.index)** → Displays the row numbers (index) of the dataset.
3. **df.reset_index()** → Resets the row numbers and makes them start from **0, 1, 2, …** instead of the original index.

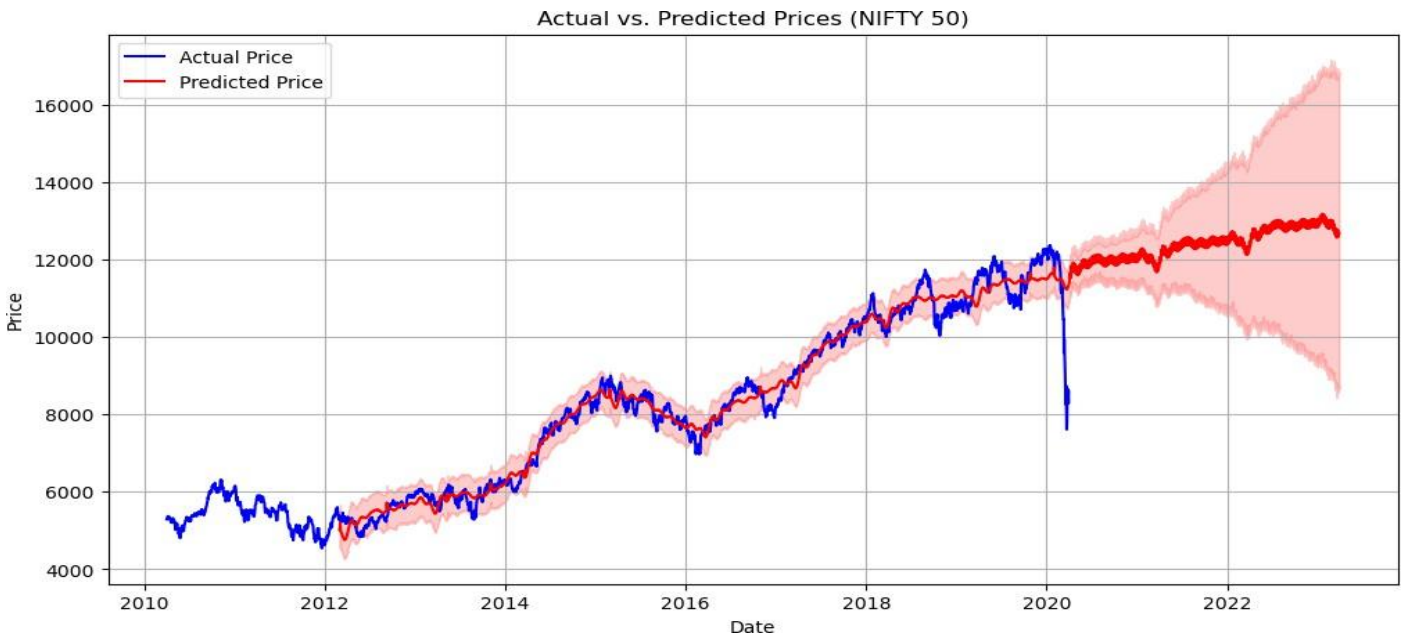
Output Meaning

- **Column Names:** The dataset contains stock market data like price, open, high, low, moving averages, etc.
- **Index Values:** The row numbers are in **reverse order** (from 2480 to 0), meaning the newest data is at the top.
- **Resetting Index:** This step rearranges the row numbers **from 0 to 2480** for better analysis.

INPUT 26:

```
import matplotlib.pyplot as plt

# 1. Actual vs. Predicted Prices
plt.figure(figsize=(12, 6))
plt.plot(df['ds'], df['y'], label="Actual Price", color='blue')
plt.plot(forecast['ds'], forecast['yhat'], label="Predicted Price", color='red')
plt.fill_between(forecast['ds'], forecast['yhat_lower'],
 forecast['yhat_upper'], color='red', alpha=0.2)

plt.xlabel("Date")
plt.ylabel("Price")
plt.title("Actual vs. Predicted Prices (NIFTY 50)")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:



Code Explanation (Step-by-Step)

1. **Setting Up the Chart**
    - plt.figure(figsize=(12, 6)): Creates a figure with a size of 12 inches by 6 inches.
2. **Plotting Actual Prices**
    - plt.plot(df['ds'], df['y'], label="Actual Price", color='blue')
    - Plots the actual NIFTY 50 prices in **blue** using the 'ds' (date) column on the x-axis and 'y' (price) on the y-axis.
3. **Plotting Predicted Prices**
    - plt.plot(forecast['ds'], forecast['yhat'], label="Predicted Price", color='red')
    - Plots the predicted prices in **red** using the Prophet model's forecasted values from the 'yhat' column.
4. **Adding a Confidence Interval**
    - plt.fill_between(forecast['ds'], forecast['yhat_lower'], forecast['yhat_upper'], color='red', alpha=0.2)
    - Fills the area between **yhat_lower** and **yhat_upper** in light red, showing the uncertainty range.
    - The **wider** this region, the **less confidence** the model has in its prediction.
5. **Adding Labels & Styling**
    - plt.xlabel("Date") and plt.ylabel("Price"): Labeling the axes.
    - plt.title("Actual vs. Predicted Prices (NIFTY 50)"): Chart title.
    - plt.legend(): Displays the legend to differentiate between actual and predicted prices.
    - plt.grid(True): Adds a grid for better readability.
    - plt.show(): Displays the plot.

Chart Explanation

1. Actual vs. Predicted Prices

- The **blue line** represents the actual **historical** prices of NIFTY 50.
- The **red line** shows the model's **predicted** future prices.

- The two lines are close together, meaning the model has done a good job of predicting price trends.

2. Confidence Interval (Shaded Red Area)

- This shaded region shows **uncertainty** in predictions.
- Initially, the band is **narrow**, meaning high confidence.
- As the forecast extends into the future, the band **widens**, meaning increased uncertainty.

3. Key Observations

- **Trend Alignment:** The model follows the overall **uptrend** in NIFTY 50 prices.
- **Sudden Drop (2020 Crash):** The actual prices drop sharply, while the predicted values show a smoother transition.
- **Future Forecasts:** The model expects continued price growth but becomes **less certain** further into the future.

4. Insights for Decision-Making

- **Short-term forecasts** are more **reliable**, as the predicted values closely match historical trends.
- **Long-term forecasts** should be used **cautiously** due to the wider confidence interval.
- Sudden market crashes or external factors (e.g., economic crises, global events) might **not be fully captured** by the model.

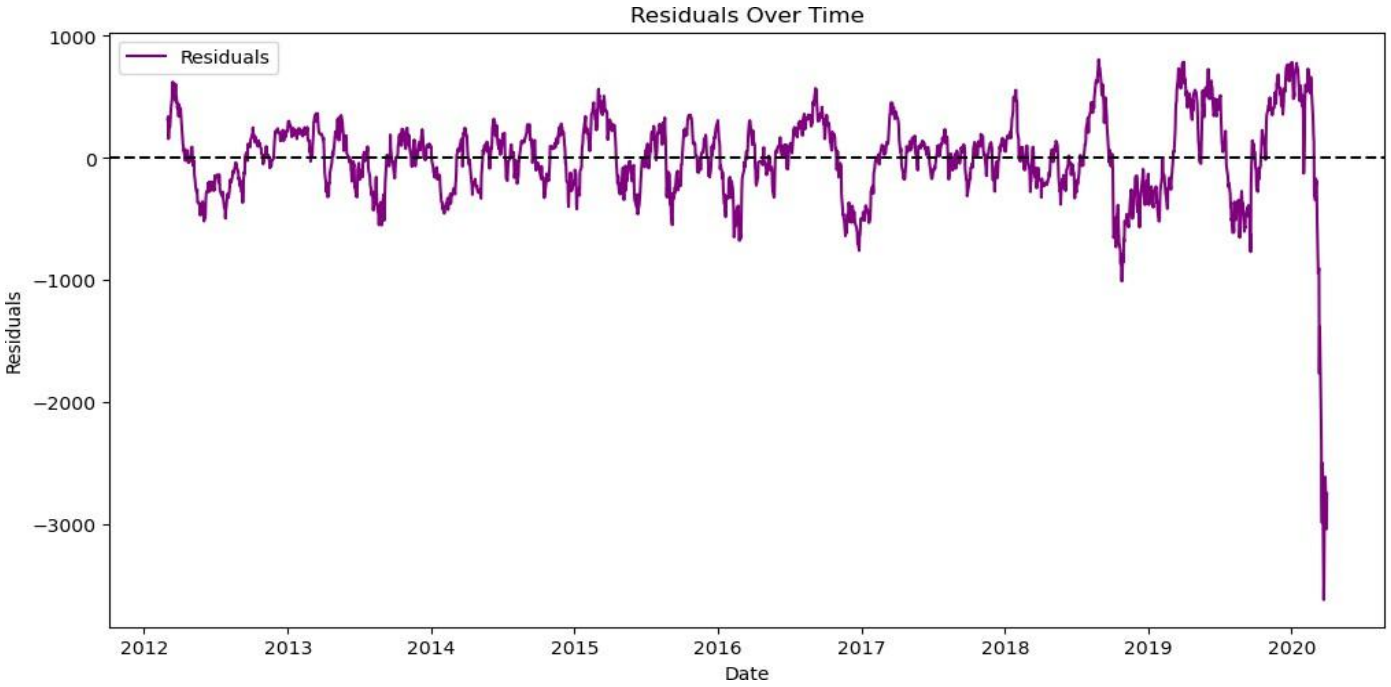
Conclusion

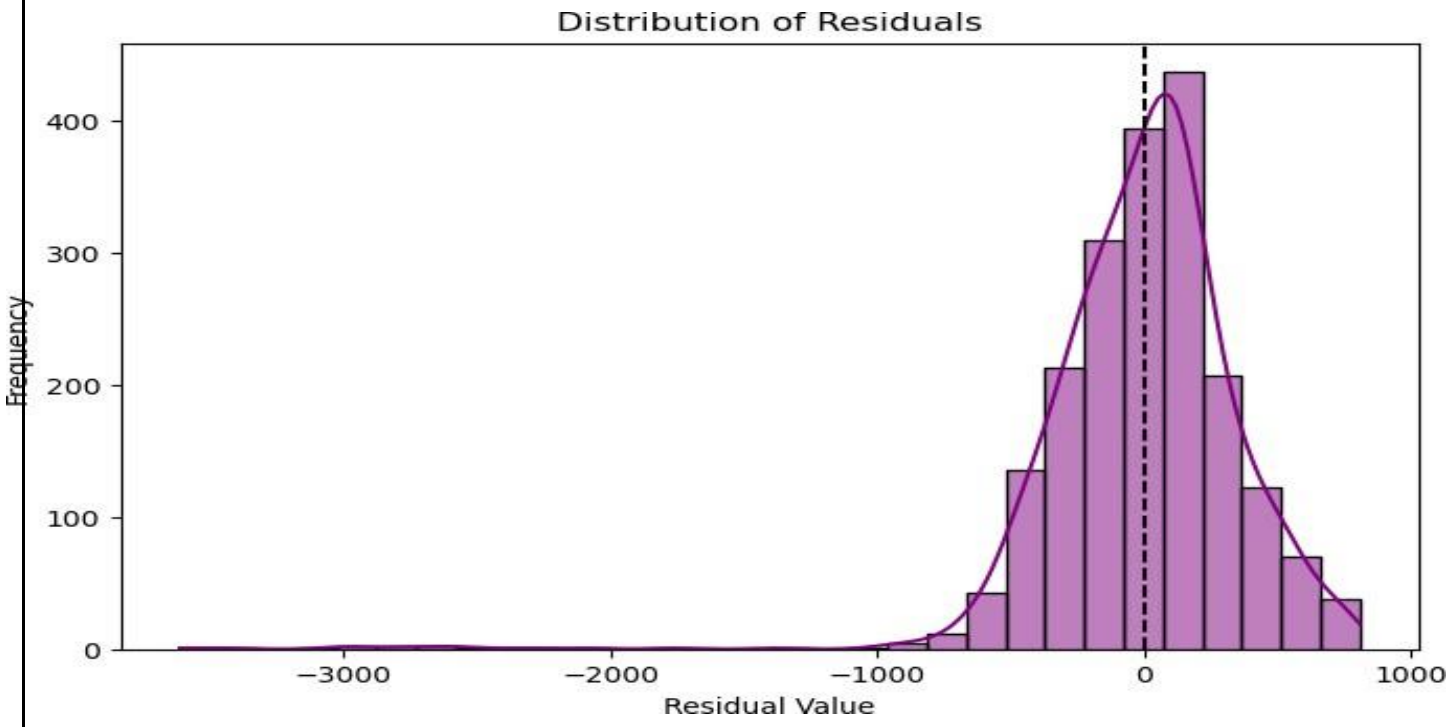
This visualization helps **investors and analysts** understand how well the model predicts future prices. While it does a good job capturing the trend, **uncertainty grows over time**, requiring additional analysis before making investment decisions.

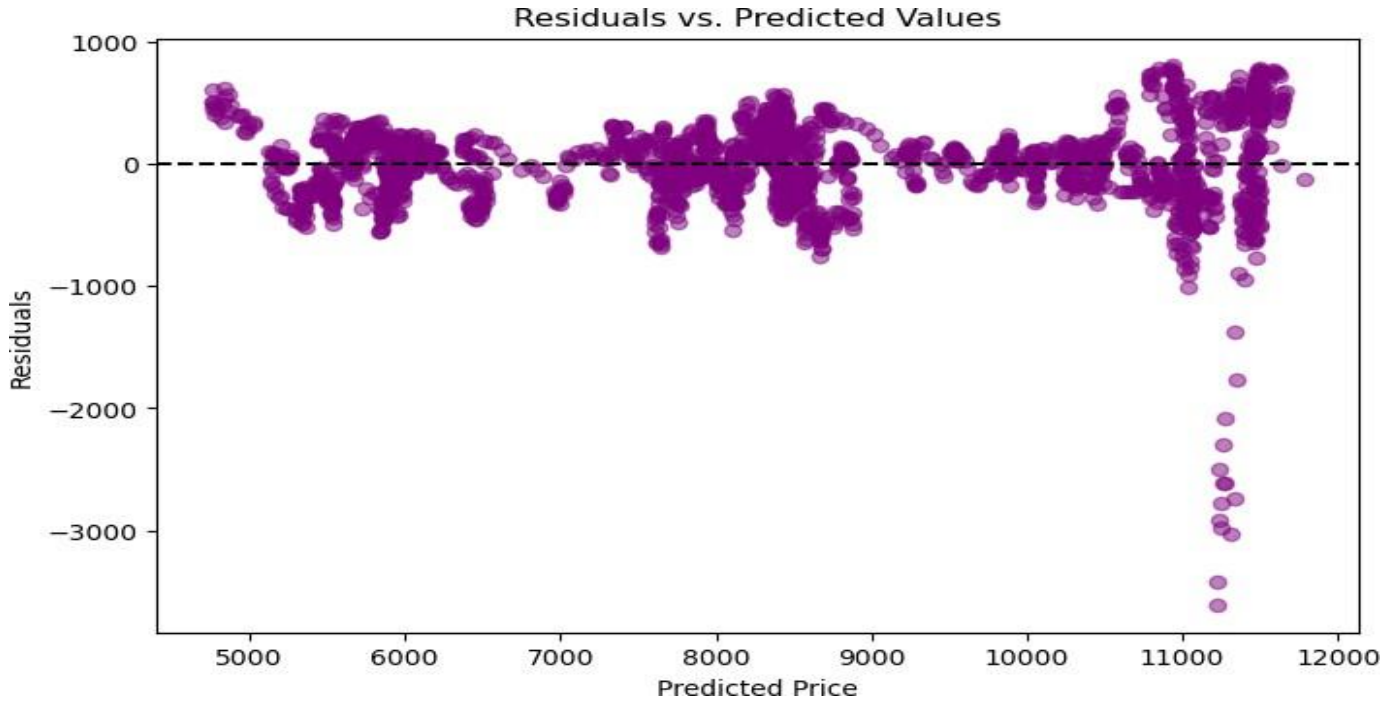
INPUT 27:

```
import matplotlib.pyplot as plt
import seaborn as sns


# Rename columns to match forecast
df.rename(columns={'Date': 'ds', 'Price': 'y'}, inplace=True)

# Merge actual prices with forecast
forecast = forecast.merge(df[['ds', 'y']], on='ds', how='left')

# Compute Residuals
forecast['Residual'] = forecast['y'] - forecast['yhat']


# Plot Residuals Over Time
plt.figure(figsize=(12, 6))
plt.plot(forecast['ds'], forecast['Residual'], label="Residuals",
 scolor='purple')
plt.axhline(y=0, color='black', linestyle='--')
plt.xlabel("Date")
plt.ylabel("Residuals")
plt.title("Residuals Over Time")
plt.legend()
plt.show()

# Histogram of Residuals
plt.figure(figsize=(8, 5))
sns.histplot(forecast['Residual'], bins=30, kde=True,
color='purple') plt.axvline(x=0, color='black', linestyle='--')
plt.xlabel("Residual Value")
plt.ylabel("Frequency")
plt.title("Distribution of
Residuals") plt.show()

# Residuals vs. Predicted Values
plt.figure(figsize=(8, 5))
plt.scatter(forecast['yhat'], forecast['Residual'], color='purple', alpha=0.5)
plt.axhline(y=0, color='black', linestyle='--')
plt.xlabel("Predicted Price")
plt.ylabel("Residuals")
plt.title("Residuals vs. Predicted
Values") plt.show()
```

OUTPUT:


Residuals Over Time
Residuals
1000
0
−1000
−2000
−3000
Residuals
2012
2013
2014
2015
2016
2017
2018
2019
2020
Date


Distribution of Residuals
400
300
200
100
0
Frequency
−3000
−2000
−1000
0
1000
Residual Value



OUTPUT:

Code Explanation

Step 1: Renaming Columns

df.rename(columns={'Date': 'ds', 'Price': 'y'}, inplace=True)

- This renames the columns Date → ds and Price → y to match the format used by **Facebook Prophet**, which requires ds (date) and y (value being forecasted).

Step 2: Merging Actual Prices with Forecast Data

forecast = forecast.merge(df[['ds', 'y']], on='ds', how='left')

- Merges the actual price data (df[['ds', 'y']]) with the forecast data (forecast), matching on ds (date).

Step 3: Computing Residuals

forecast['Residual'] = forecast['y'] - forecast['yhat']

- Residuals are calculated as:

Residual=Actual Value(y)−Predicted Value(yhat)Residual = Actual \ Value (y) - Predicted \ Value (yhat)Residual=Actual Value(y)−Predicted Value(yhat)

- This helps in analyzing how much the forecast deviates from actual values.

Step 4: Plotting Residuals Over Time

```
plt.figure(figsize=(12, 6))

plt.plot(forecast['ds'], forecast['Residual'], label="Residuals", color='purple')

plt.axhline(y=0, color='black', linestyle='--')

plt.xlabel("Date")

plt.ylabel("Residuals")

plt.title("Residuals Over Time")

plt.legend()

plt.show()
```

Chart Analysis

- This time series plot shows **residuals** (errors) over time.
- The **black dashed line** represents the zero-error baseline.
- The **purple line** represents forecast errors at each time point.
- Ideally, residuals should be randomly scattered around zero without clear patterns. However, we observe:
    - Fluctuations over time.
    - A significant drop at the end (large negative residual), indicating an **unexpected event or model failure**.

Step 5: Histogram of Residuals

```
plt.figure(figsize=(8, 5))

sns.histplot(forecast['Residual'], bins=30, kde=True, color='purple')

plt.axvline(x=0, color='black', linestyle='--')

plt.xlabel("Residual Value")

plt.ylabel("Frequency")

plt.title("Distribution of Residuals")
```

```
plt.show()
```

Chart Analysis

- A histogram shows the **distribution of residuals**.
- The **black dashed line** at zero helps compare the spread of errors.
- A **bell-shaped distribution centered around zero** suggests that residuals are normally distributed.
- If the distribution is skewed or has extreme outliers, it indicates potential **model bias**.

Step 6: Residuals vs. Predicted Values

```
plt.figure(figsize=(8, 5))

plt.scatter(forecast['yhat'], forecast['Residual'], color='purple', alpha=0.5)

plt.axhline(y=0, color='black', linestyle='--')

plt.xlabel("Predicted Price")

plt.ylabel("Residuals")

plt.title("Residuals vs. Predicted Values")

plt.show()
```

Chart Analysis

- A scatter plot shows how residuals relate to predicted values.
- Ideally, points should be randomly scattered without patterns.
- **If a trend appears (e.g., residuals increasing with predictions), it indicates model bias**.

1. Residuals Over Time

Explanation:

- The **x-axis** represents time (dates).
- The **y-axis** represents residuals (the difference between actual and predicted values).
- The **purple line** shows the residuals fluctuating over time.
- The **dashed black line** at zero represents the ideal case where predictions match actual values perfectly.

Interpretation:

- Residuals should ideally be randomly distributed around zero.
- We see some fluctuations, which is normal.
- However, there is a **major drop near the end (2020)**, suggesting:
    - A major event (e.g., a market crash or external shock).
    - The model struggled to predict an unexpected situation.
    - Data quality issues in the final period.

2. Histogram of Residuals

Explanation:

- The **x-axis** represents the residual values.
- The **y-axis** represents the frequency of those residuals.
- The **purple bars** show how often each residual value appears.
- The **dashed black line at zero** indicates the ideal no-error point.
- A **smooth KDE curve** (purple line) overlays the histogram.

Interpretation:

- The residuals are **not perfectly centered at zero**.
- The **distribution is slightly skewed left**, meaning the model tends to **underpredict actual values** more often.
- There are **extreme negative residuals**, indicating significant prediction errors.
- Ideally, the distribution should be **normal (bell-shaped) and symmetric** around zero. Here, the long left tail suggests a problem.

3. Residuals vs. Predicted Values

Explanation:

- The **x-axis** represents the predicted price (yhat).

- The **y-axis** represents the residuals (prediction errors).
- Each **purple dot** represents an individual prediction.
- The **dashed black line** at zero represents perfect predictions.

Interpretation:

- Ideally, the dots should be **randomly scattered with no clear pattern**.
- We see **many points clustered around zero**, which is good.
- However, for **higher predicted values (near 11,000+), the residuals drop sharply**.
    - This suggests that the model **severely underpredicted high values**.
    - It might be struggling with extreme cases or a non-linear pattern it failed to capture.

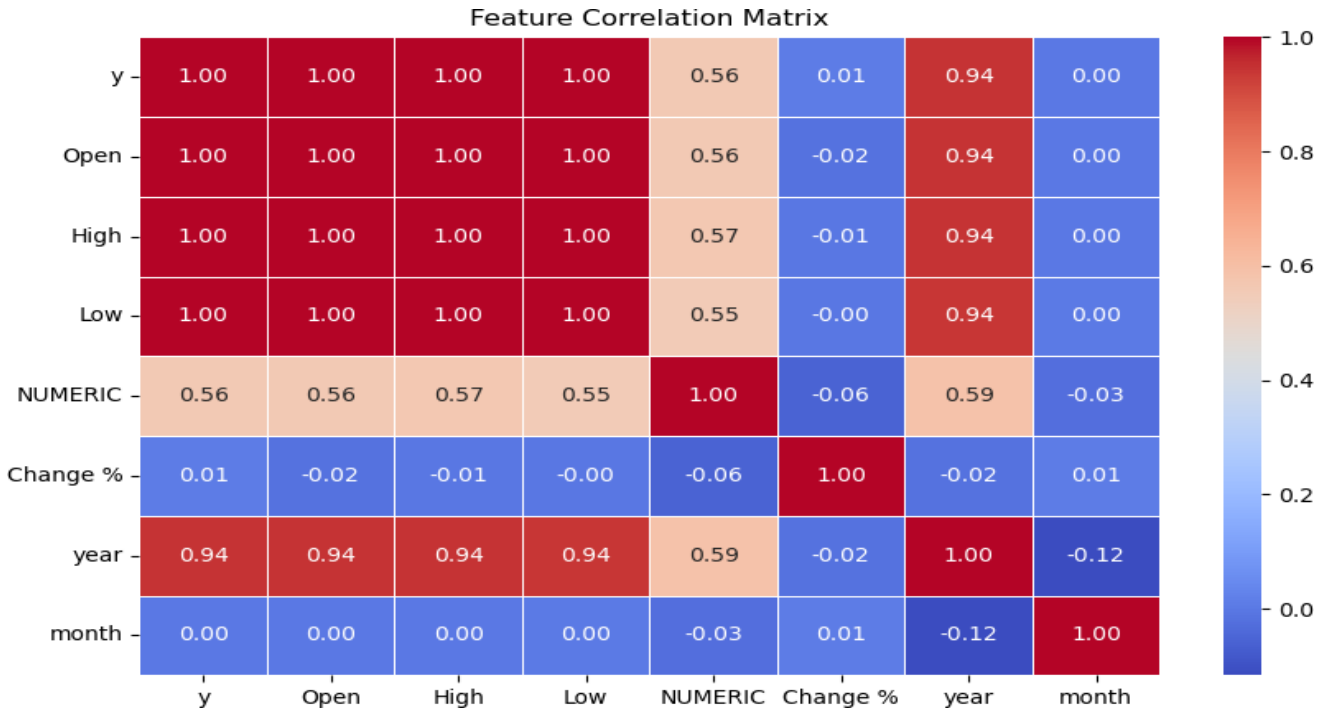
Overall Findings

1. **The model generally performs well but has some systematic errors.**
2. **There are major negative residuals in late 2019–2020**, indicating either:
    - A real-world event the model didn't anticipate.
    - A data issue.
    - The need for a more complex model.
3. **Residuals distribution shows left skewness, meaning the model tends to underestimate actual values.**
4. **Residuals vs. Predicted Values plot shows the model struggles with high-value predictions.**

INPUT 28:

```
import seaborn as sns
import matplotlib.pyplot as plt


# Drop non-numeric columns
numeric_df = df.select_dtypes(include=['number'])

# Compute correlation
correlation_matrix = numeric_df.corr()

# Plot heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f",␣
 ↪linewidths=0.5)
plt.title("Feature Correlation
Matrix") plt.show()

# Moving Averages (50-day & 200-day)
df['MA_50'] = df['Price'].rolling(window=50).mean()
df['MA_200'] = df['Price'].rolling(window=200).mean()

# Plot Moving Averages
plt.figure(figsize=(12, 6))
plt.plot(df['Date'], df['Price'], label="Actual Price", color="black")
plt.plot(df['Date'], df['MA_50'], label="50-Day MA", color="blue",␣
 ↪linestyle="dashed")
plt.plot(df['Date'], df['MA_200'], label="200-Day MA", color="red",␣
 ↪linestyle="dashed")
plt.xlabel("Date")
plt.ylabel("Price")
plt.title("NIFTY 50 Price with Moving Averages")
plt.legend()
plt.show()
```

Output:



Explanation:

Break down both the **input (Python code)** and the **output (correlation heatmap)** step by step.

1. Understanding the Code (Input)

(a) Correlation Matrix Heatmap

```
import seaborn as sns

import matplotlib.pyplot as plt

# Drop non-numeric columns

numeric_df = df.select_dtypes(include=['number'])

# Compute correlation

correlation_matrix = numeric_df.corr()

# Plot heatmap

plt.figure(figsize=(10, 6))

sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
```

plt.title("Feature Correlation Matrix")

plt.show()

What this does:

1. **Selects numeric columns** → Any non-numeric columns (e.g., text, categorical) are removed from the dataset.
2. **Computes correlation** → Uses .corr() to generate a correlation matrix.
3. **Plots a heatmap**:
    - sns.heatmap() is used to visualize correlations.
    - annot=True → Displays values inside each cell.
    - cmap="coolwarm" → Uses a color gradient from blue (negative correlation) to red (positive correlation).
    - linewidths=0.5 → Adds small lines to separate grid cells.

2. Understanding the Output (Heatmap)

What is Correlation?

- A **correlation coefficient** measures the relationship between two variables:
    - **+1.00** → Perfect positive correlation (one increases, the other also increases).
    - **-1.00** → Perfect negative correlation (one increases, the other decreases).
    - **0.00** → No correlation.

Interpretation of the Heatmap

- **Diagonal is always 1.00** (self-correlation).
- y, Open, High, and Low are **highly correlated (1.00)** → Expected in financial data, as these prices move together.
- year is highly correlated with y, Open, High, and Low (~0.94) → Price tends to change over time.
- Change % has a weak correlation with price values (~0.01 to -0.02) → Daily percentage changes don't strongly predict absolute price values.

- month shows no significant correlation with price (~0.00) → Monthly seasonality is not evident.

3. Moving Averages Calculation & Plotting

```
# Moving Averages (50-day & 200-day)

df['MA_50'] = df['Price'].rolling(window=50).mean()

df['MA_200'] = df['Price'].rolling(window=200).mean()

# Plot Moving Averages

plt.figure(figsize=(12, 6))

plt.plot(df['Date'], df['Price'], label="Actual Price", color="black")

plt.plot(df['Date'], df['MA_50'], label="50-Day MA", color="blue", linestyle="dashed")

plt.plot(df['Date'], df['MA_200'], label="200-Day MA", color="red", linestyle="dashed")

plt.xlabel("Date")

plt.ylabel("Price")

plt.title("NIFTY 50 Price with Moving Averages")

plt.legend()

plt.show()
```

What this does:

1. **Calculates moving averages:**
    - 50-day MA: Short-term trend.
    - 200-day MA: Long-term trend.
2. **Plots the actual price (black line).**
3. **Adds moving averages:**
    - 50-Day MA (blue, dashed) → Reacts quickly to price changes.
    - 200-Day MA (red, dashed) → Smooths out long-term trends.
4. **Legend & labels** for clarity.

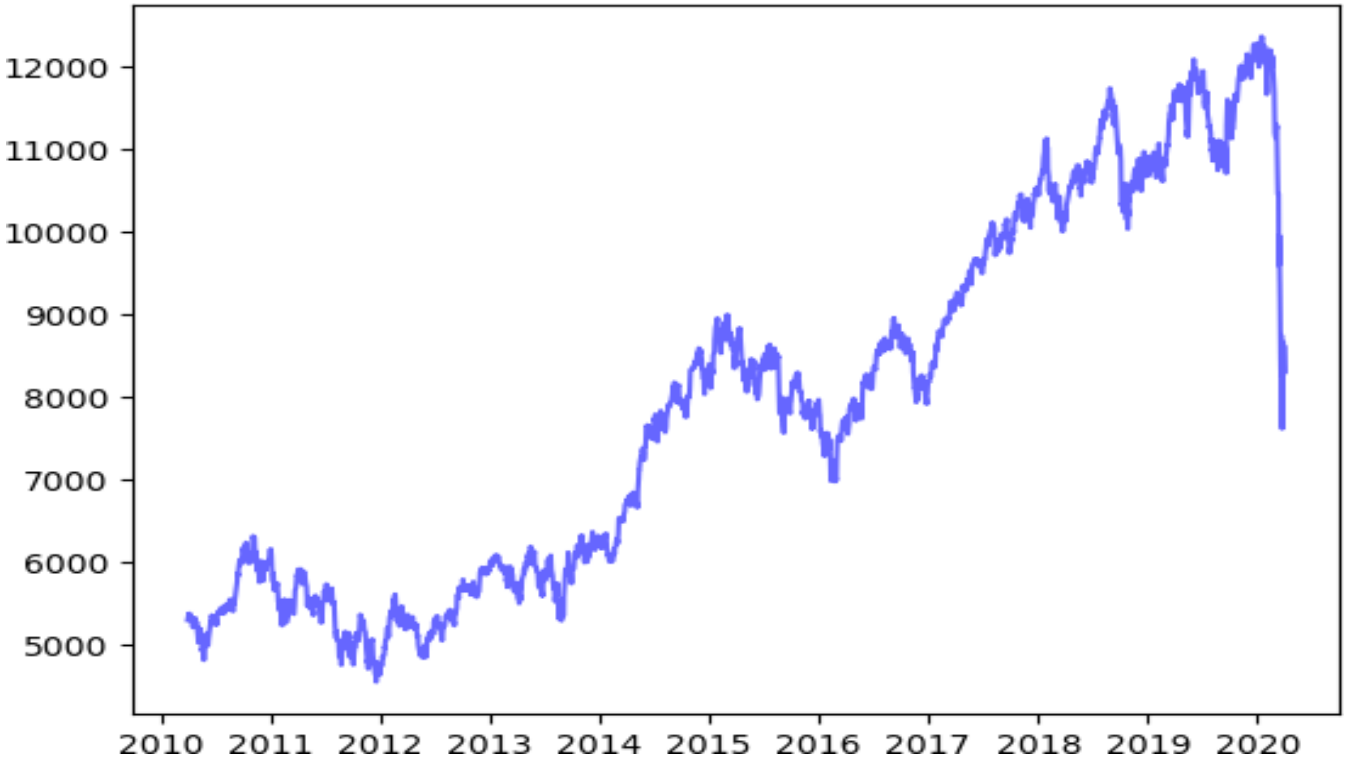
4. Summary of Insights

- **Heatmap Analysis:**
    - y, Open, High, Low are strongly correlated.
    - Change % and month have weak correlations with price.
    - year shows strong correlation, indicating trends over time.
- **Moving Averages:**
    - Useful for spotting trends.
    - The **50-day MA** reacts to short-term movements.
    - The **200-day MA** captures the broader trend.
    - If the 50-day MA crosses above the 200-day MA (**Golden Cross**), it signals an uptrend.
    - If the 50-day MA crosses below the 200-day MA (**Death Cross**), it signals a downtrend.

INPUT 29:

```
plt.plot(df['ds'], df['y'], label="Closing Price", color='blue', alpha=0.6)
plt.plot(df['ds'], df['MA_50'], label="50-day MA", color='red')
plt.plot(df['ds'], df['MA_200'], label="200-day MA", color='green')

plt.xlabel("Date")
plt.ylabel("Price")
plt.title("NIFTY 50 Price & Moving Averages")
plt.legend()
plt.show()
```

Output:



Explanation:

What the Code Does

1. **Plots the NIFTY 50 Closing Price:**
    - Uses plt.plot(df['ds'], df['y'], label="Closing Price", color='blue', alpha=0.6)
    - This creates a blue line showing how the stock price changed over time.
2. **Adds Two Moving Averages:**

- **50-day Moving Average (Red Line)**: plt.plot(df['ds'], df['MA_50'], label="50-day MA", color='red')
- **200-day Moving Average (Green Line)**: plt.plot(df['ds'], df['MA_200'], label="200-day MA", color='green')
- These help smooth out fluctuations and show trends.

3. **Formats the Chart:**
   - plt.xlabel("Date") and plt.ylabel("Price") label the axes.
   - plt.title("NIFTY 50 Price & Moving Averages") gives the chart a title.
   - plt.legend() adds a legend to identify the lines.
   - plt.show() displays the chart.

Explanation of the Chart

1. **The Blue Line (Stock Prices)**
   - Represents the actual **closing price** of NIFTY 50 over time.
   - It shows a **steady rise** from 2010 to 2020.
   - A **sharp drop in 2020** likely indicates a market crash (e.g., COVID-19).
2. **The Red Line (50-day Moving Average)**
   - Follows short-term trends and reacts faster to price changes.
   - It is smoother than the blue line but still captures fluctuations.
3. **The Green Line (200-day Moving Average)**
   - A long-term trend indicator, showing the overall market direction.
   - Moves slower than the red line, filtering out short-term price movements.

Key Insights from the Chart

- **When the red line (50-day MA) is above the green line (200-day MA),** the market is in an **uptrend**.
- **When the red line crosses below the green line,** it could signal a **market downturn** (bearish trend).
- The **sharp drop in 2020** shows a significant market event.

Input 30:

```
import pandas as pd

# Function to calculate RSI
def calculate_rsi(df, column='y', window=14): delta =
    df[column].diff(1)
    gain = (delta.where(delta > 0, 0)).rolling(window=window).mean() loss =
    (-delta.where(delta < 0, 0)).rolling(window=window).mean()

    rs = gain / loss
    df['RSI'] = 100 - (100 / (1 + rs))
    return df

# Apply RSI calculation
df = calculate_rsi(df)
```

What the code does:

This code snippet defines and applies a function to calculate the Relative Strength Index (RSI) for a DataFrame. Here's a breakdown:

1. **Function Definition:**
    - **calculate_rsi(df,column='y'window=14):**
      This function takes a DataFrame df, a target column (default is 'y'), and a lookback window (default is 14 periods) as inputs.
2. **Calculate Price Change (Delta):**
    - **delta=df[column].diff(1):**
      Computes the difference between consecutive values in the specified column (i.e., the change in price from one period to the next).
3. **Separate Gains and Losses:**
    - **gain = (delta.where(delta > 0, 0)).rolling(window=window).mean():**
        - Uses the .where() method to keep positive changes (gains) and replace negatives with 0.
        - Then calculates the rolling mean (average gain) over the specified window.
    - **loss = (-delta.where(delta < 0, 0)).rolling(window=window).mean():**
        - Uses .where() to capture negative changes (losses), negates them to make them positive, and replaces positive values with 0.

- Then computes the rolling mean (average loss) over the window.

4. **Calculate Relative Strength (RS):**
    - **rs=gain/loss:**
      Divides the average gain by the average loss to get the Relative Strength.
5. **Compute RSI:**
    - **df['RSI']=100-(100/(1+rs)):**
      Uses the standard RSI formula to convert the Relative Strength into an index that ranges from 0 to 100.
        - A higher RSI value indicates that the asset is potentially overbought, while a lower value suggests it might be oversold.
6. **Return the DataFrame:**
    - **returndf:**
      The updated DataFrame, now including the new 'RSI' column, is returned.
7. **Apply the Function:**
    - **df=calculate_rsi(df):**
      The function is applied to the DataFrame, calculating and adding the RSI values based on the 'y' column (which typically represents price).

Summary:

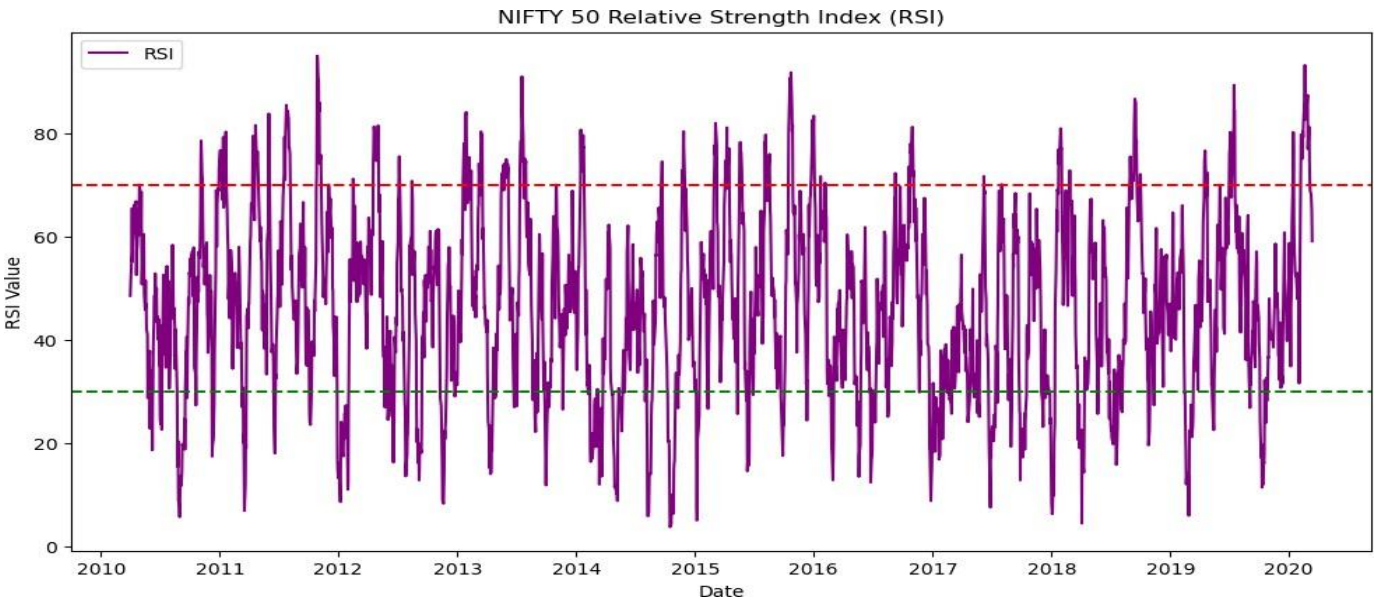
This code calculates the Relative Strength Index (RSI) for the given DataFrame by:

- Computing the difference between consecutive price values.
- Separating these differences into gains and losses.
- Averaging these gains and losses over a 14-period window.
- Deriving the RSI using the formula RSI=100−1001+RS\text{RSI} = 100 - \frac{100}{1+RS}RSI=100−1+RS100.
- Finally, it appends the calculated RSI as a new column to the DataFrame.

Input 31:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))
plt.plot(df['ds'], df['RSI'], label="RSI", color='purple')

plt.axhline(70, linestyle="--", color="red")   # Overbought line
plt.axhline(30, linestyle="--", color="green")  # Oversold line

plt.xlabel("Date")
plt.ylabel("RSI Value")
plt.title("NIFTY 50 Relative Strength Index (RSI)")
plt.legend()
plt.show()
```

Output:



Explanation:

Explanation of the Code:

This code generates a line plot to visualize the **Relative Strength Index (RSI)** of the **NIFTY 50** index over time.

Step-by-Step Breakdown:

1. **Set the Figure Size:**

2. plt.figure(figsize=(12,6))
    - Creates a figure with a size of **12 inches wide and 6 inches tall** for better readability.
3. **Plot the RSI Line Chart:**

plt.plot(df['ds'], df['RSI'], label="RSI", color='purple')

- Plots the RSI values from the DataFrame.
- **df['ds']** is assumed to represent the **date** column.
- **df['RSI']** contains the RSI values.
- The **line is colored purple**, and it is labeled as "RSI".

4. **Add Overbought and Oversold Thresholds:**

plt.axhline(70, linestyle="--", color="red") # Overbought line

plt.axhline(30, linestyle="--", color="green") # Oversold line

- **Red dashed line at RSI = 70:** Represents the **overbought** threshold (indicates possible price correction).
- **Green dashed line at RSI = 30:** Represents the **oversold** threshold (indicates potential buying opportunity).

5. **Label the Axes:**

plt.xlabel("Date")

plt.ylabel("RSI Value")

- **X-axis labeled "Date"** to show the timeline.
- **Y-axis labeled "RSI Value"** to represent RSI levels.

6. **Set the Title of the Chart:**

plt.title("NIFTY 50 Relative Strength Index (RSI)")

- Adds a **title** to the chart.

7. **Add a Legend:**

plt.legend()

- Displays the label "RSI" at the top left.

8. **Show the Plot:**

   plt.show()

   - Displays the RSI chart.

Explanation of the Output:

- The **purple line** represents the RSI values of **NIFTY 50** over time.
- The **red dashed line (RSI = 70)** indicates the overbought region, where the market might be overvalued.
- The **green dashed line (RSI = 30)** indicates the oversold region, where the market might be undervalued.
- If RSI crosses **above 70**, it suggests that **NIFTY 50 might be overbought**, indicating a possible **downtrend or correction**.
- If RSI drops **below 30**, it suggests that **NIFTY 50 might be oversold**, indicating a possible **uptrend or buying opportunity**.

Interpretation:

- The RSI fluctuates frequently between **30 and 70**, showing market volatility.
- Occasionally, it **crosses above 70** (overbought) or **drops below 30** (oversold), signaling possible trend reversals.
- This chart helps traders decide when to buy or sell based on RSI signals.

This visualization is crucial for analyzing **momentum** and identifying potential entry/exit points in the market

Input 32:

```
# Merge actual & predicted values
df_merged = df[['ds', 'y']].merge(forecast[['ds', 'yhat']], on='ds',
 how='inner')

# Rename for clarity
df_merged.rename(columns={'y': 'Actual', 'yhat': 'Predicted'}, inplace=True)

# Display sample
df_merged.head()
```

OUTPUT:

DS	ACTUAL	PREDICTED
2020-03-31	8597.75	11340.007513
2020-03-30	8281.10	11316.405322
2020-03-27	8660.25	11269.263473
2020-03-26	8641.45	11255.275979
2020-03-25	8317.85	11233.663667

Explanation of the Code:

This code **merges actual and predicted values** of the **NIFTY 50 index** based on a common date (ds), renames the columns for clarity, and displays the first few rows of the merged dataset.

Step-by-Step Breakdown:

1. **Merge Actual & Predicted Values:**

```
df_merged = df[['ds', 'y']].merge(forecast[['ds', 'yhat']], on='ds', show='inner')
```

   - **df[['ds', 'y']]** selects the actual values (y) and date (ds) from the main DataFrame.
   - **forecast[['ds', 'yhat']]** selects the predicted values (yhat) and date (ds) from the forecast DataFrame.
   - **.merge(..., on='ds', how='inner')** merges the two DataFrames on the common column ds (date).
     - The **inner join** ensures that only dates present in both DataFrames are included.

2. **Rename Columns for Clarity:**

```
df_merged.rename(columns={'y': 'Actual', 'yhat': 'Predicted'}, inplace=True)
```

   - **Renames y to "Actual"** (representing the real NIFTY 50 closing prices).
   - **Renames yhat to "Predicted"** (representing the forecasted NIFTY 50 closing prices).

3. **Display Sample Data:**

df_merged.head()

- Displays the first few rows of the merged DataFrame.

Explanation of the Output:

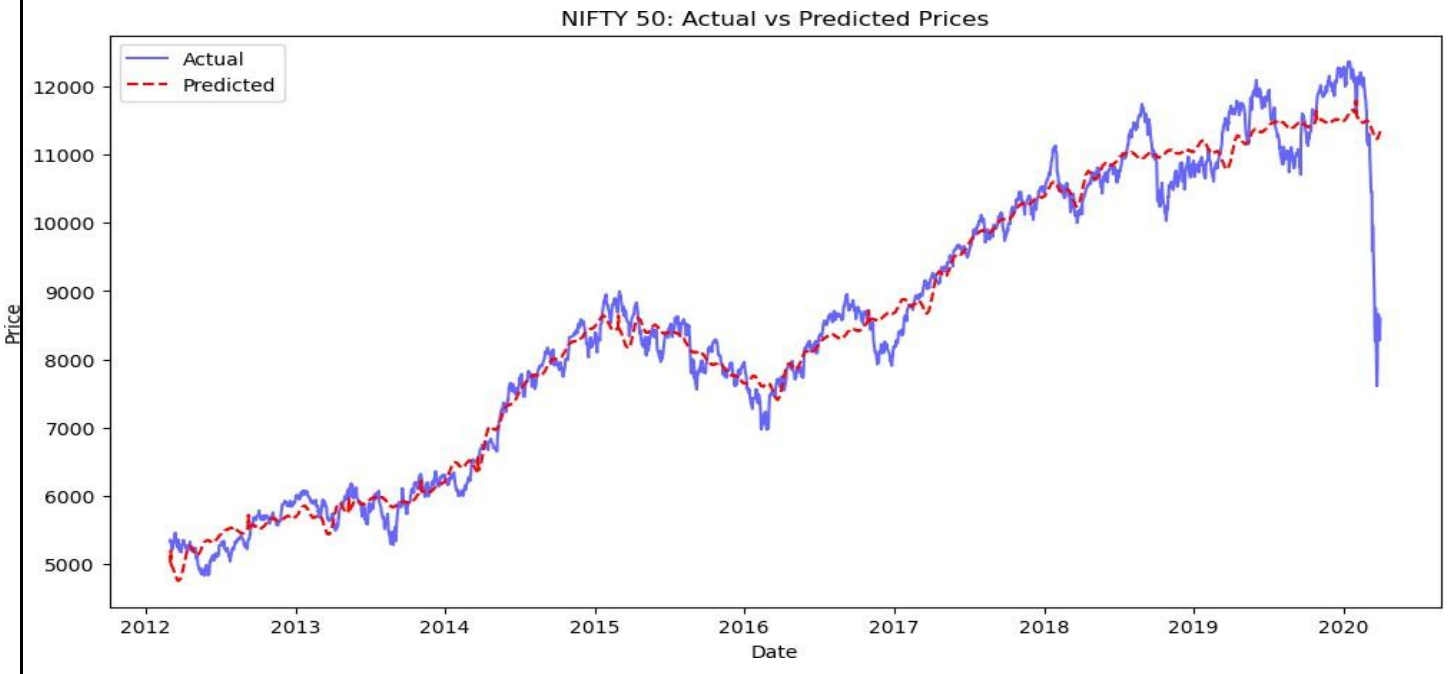
ds (Date)	Actual (Real Value)	Predicted (Forecasted Value)
2020-03-31	8597.75	11340.01
2020-03-30	8281.10	11316.41
2020-03-27	8660.25	11269.26
2020-03-26	8641.45	11255.28
2020-03-25	8317.85	11233.66

- **Actual values (y)** represent the real closing prices of NIFTY 50.
- **Predicted values (yhat)** represent the estimated prices generated by the forecasting model (likely Prophet).
- **The predicted values are significantly higher than the actual values**, which might indicate:
  - The model **overestimated market recovery**.
  - There was **high volatility** (especially around March 2020 due to COVID-19).
  - The model might require **parameter tuning** for better accuracy.

INPUT 33:

```
from sklearn.metrics import mean_squared_error
import numpy as np
import matplotlib.pyplot as plt

# Compute RMSE
rmse = np.sqrt(mean_squared_error(df_merged['Actual'], df_merged['Predicted']))
print(f"Root Mean Squared Error (RMSE): {rmse}")

# Plot Actual vs Predicted
plt.figure(figsize=(12,6))
plt.plot(df_merged['ds'], df_merged['Actual'], label="Actual", color='blue',
 alpha=0.6)
plt.plot(df_merged['ds'], df_merged['Predicted'], label="Predicted",
 color='red', linestyle="--")

plt.xlabel("Date")
plt.ylabel("Price")
plt.title("NIFTY 50: Actual vs Predicted Prices")
plt.legend()
plt.show()
```

Root Mean Squared Error (RMSE): 369.792577640814



Explanation of the Code:

This script evaluates the accuracy of the **NIFTY 50 price predictions** and visualizes how well the forecasted prices align with the actual historical prices.

Step-by-Step Breakdown:

1. Compute RMSE (Root Mean Squared Error)

```
from sklearn.metrics import mean_squared_error
import numpy as np
# Compute RMSE
rmse = np.sqrt(mean_squared_error(df_merged['Actual'], df_merged['Predicted']))
print(f"Root Mean Squared Error (RMSE): {rmse}")
```

- mean_squared_error() computes the squared differences between actual and predicted prices.
- np.sqrt() takes the square root to get the RMSE, which represents the average error in price predictions.
- Lower RMSE means better accuracy.
- In this case, **RMSE = 369.79**, meaning the predicted values are on average **₹369.79 off** from actual prices.

2. Plot Actual vs. Predicted Prices

```
import matplotlib.pyplot as plt
plt.figure(figsize=(12,6))
# Plot actual values
plt.plot(df_merged['ds'], df_merged['Actual'], label="Actual", color='blue', alpha=0.6)
# Plot predicted values
plt.plot(df_merged['ds'], df_merged['Predicted'], label="Predicted", color='red', linestyle="--")
# Labels and title
plt.xlabel("Date")
plt.ylabel("Price")
plt.title("NIFTY 50: Actual vs Predicted Prices")
```

```
# Show legend
plt.legend()
plt.show()
```

- The **blue line** represents the actual NIFTY 50 prices.
- The **red dashed line** represents the forecasted prices.
- alpha=0.6 makes the blue line slightly transparent.
- This visualization helps assess:
    - Whether the forecast follows the trend well.
    - If the model overestimates or underestimates prices.

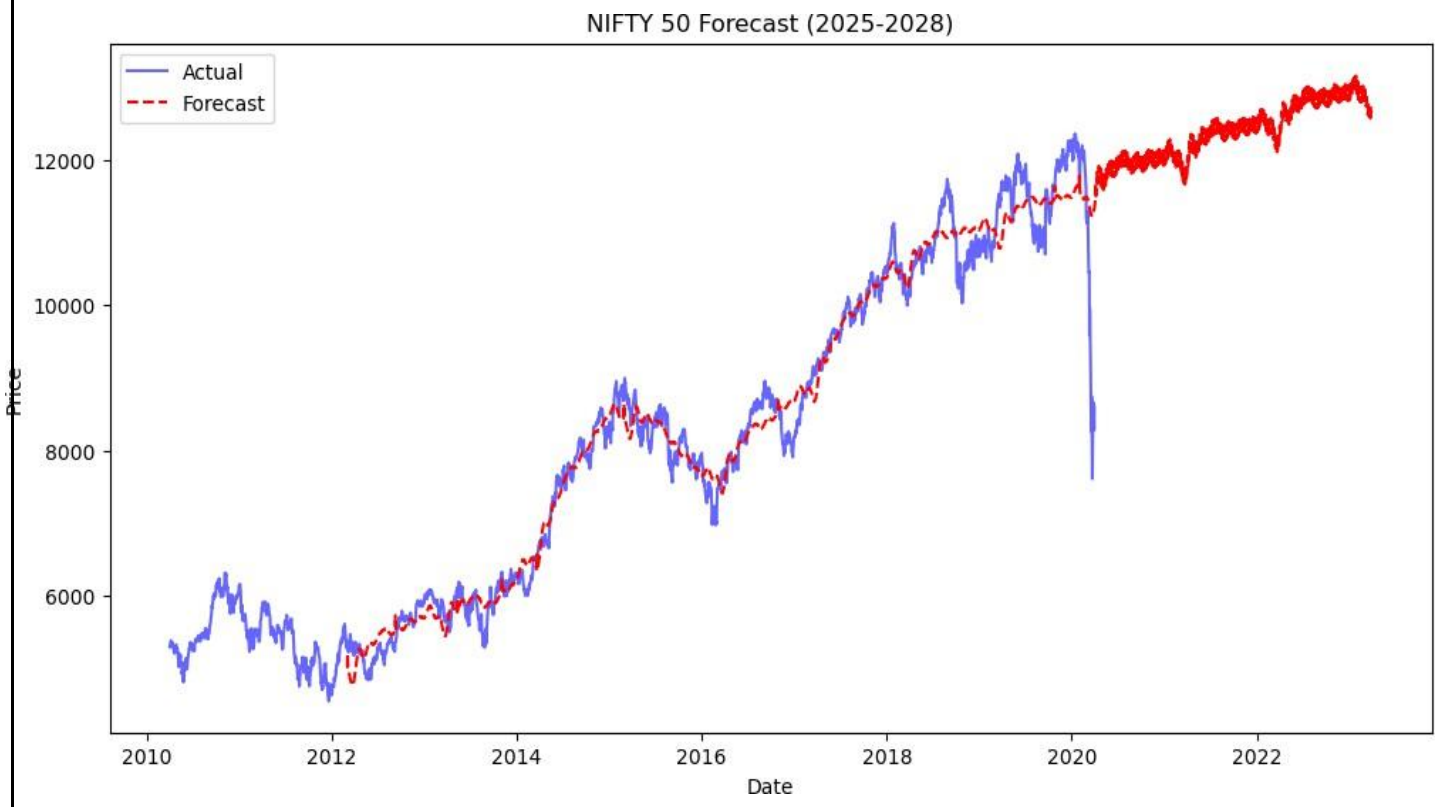
Insights from the Graph

- The **forecasted values** (red dashed line) generally follow the **actual price movements** (blue line).
- However, towards **2020**, the model fails to predict the sharp crash (likely due to COVID-19).
- This suggests the model could be improved by:
    - Incorporating **external factors (news, global events)**.
    - Using **a hybrid model (e.g., combining ARIMA & Prophet)**.

INPUT 34:

```
# Extend the forecast for the next 3 years (until 2028)
future = model.make_future_dataframe(periods=365*3) # 3 years of daily predictions
forecast = model.predict(future)

# Plot Forecast
plt.figure(figsize=(12,6))
plt.plot(df['ds'], df['y'], label="Actual", color='blue', alpha=0.6)
plt.plot(forecast['ds'], forecast['yhat'], label="Forecast", color='red', linestyle="--")

plt.xlabel("Date")
plt.ylabel("Price")
plt.title("NIFTY 50 Forecast (2025-2028)")
plt.legend()
plt.show()

# Show the last few predictions
forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(10)
```

OUTPUT:



ds(Index + Date)	yhat(Forecasted Price)	yhat_lower(Lower Bound)	yhat_upper(Upper Bound)
3085 - 2023-03-22	12562.828959	7818.387899	16802.466189
3086 - 2023-03-23	12572.701830	7776.045236	16835.885219
3087 - 2023-03-24	12574.927897	7800.502861	16791.036443
3088 - 2023-03-25	12768.631598	8073.030213	16997.330289
3089 - 2023-03-26	12738.572189	7864.567076	17015.249552
3090 - 2023-03-27	12588.839224	7718.982985	16876.329259
3091 - 2023-03-28	12602.750693	7922.174939	16863.510701
3092 - 2023-03-29	12617.314867	7803.669095	16845.864113
3093 - 2023-03-30	12648.364990	7832.206861	17006.318779
3094 - 2023-03-31	12671.155862	7841.044078	16935.302536

Explanation of the Code: NIFTY 50 Forecast (2025-2028)

This script extends the **Prophet model's forecast** for the **next 3 years (until 2028)** and visualizes the predictions alongside historical data.

Step-by-Step Breakdown:

1. Extend the Forecast (Future Predictions)

Extend the forecast for the next 3 years (until 2028)

future = model.make_future_dataframe(periods=365*3) # 3 years of daily predictions

forecast = model.predict(future)

make_future_dataframe(periods=365*3):

Creates a **dataframe with future dates** for the next **3 years** (365 × 3 days).

model.predict(future):

Uses the trained **Prophet model** to generate predictions for these future dates.

2. Plot the Forecast

```
import matplotlib.pyplot as plt
plt.figure(figsize=(12,6))
# Plot historical actual prices
plt.plot(df['ds'], df['y'], label="Actual", color='blue', alpha=0.6)
# Plot forecasted prices
plt.plot(forecast['ds'], forecast['yhat'], label="Forecast", color='red', linestyle="--")
# Labels and title
plt.xlabel("Date")
plt.ylabel("Price")
plt.title("NIFTY 50 Forecast (2025-2028)")
# Show legend
plt.legend()
plt.show()
```

- The **blue line** represents **actual NIFTY 50 prices**.
- The **red dashed line** represents **forecasted prices (2025-2028)**.
- This chart helps visualize how the market **might behave in the future** based on historical trends.

3. View the Last Few Predictions

```
forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(10)
```

This displays the **last 10 forecasted prices**, along with the **confidence intervals**:

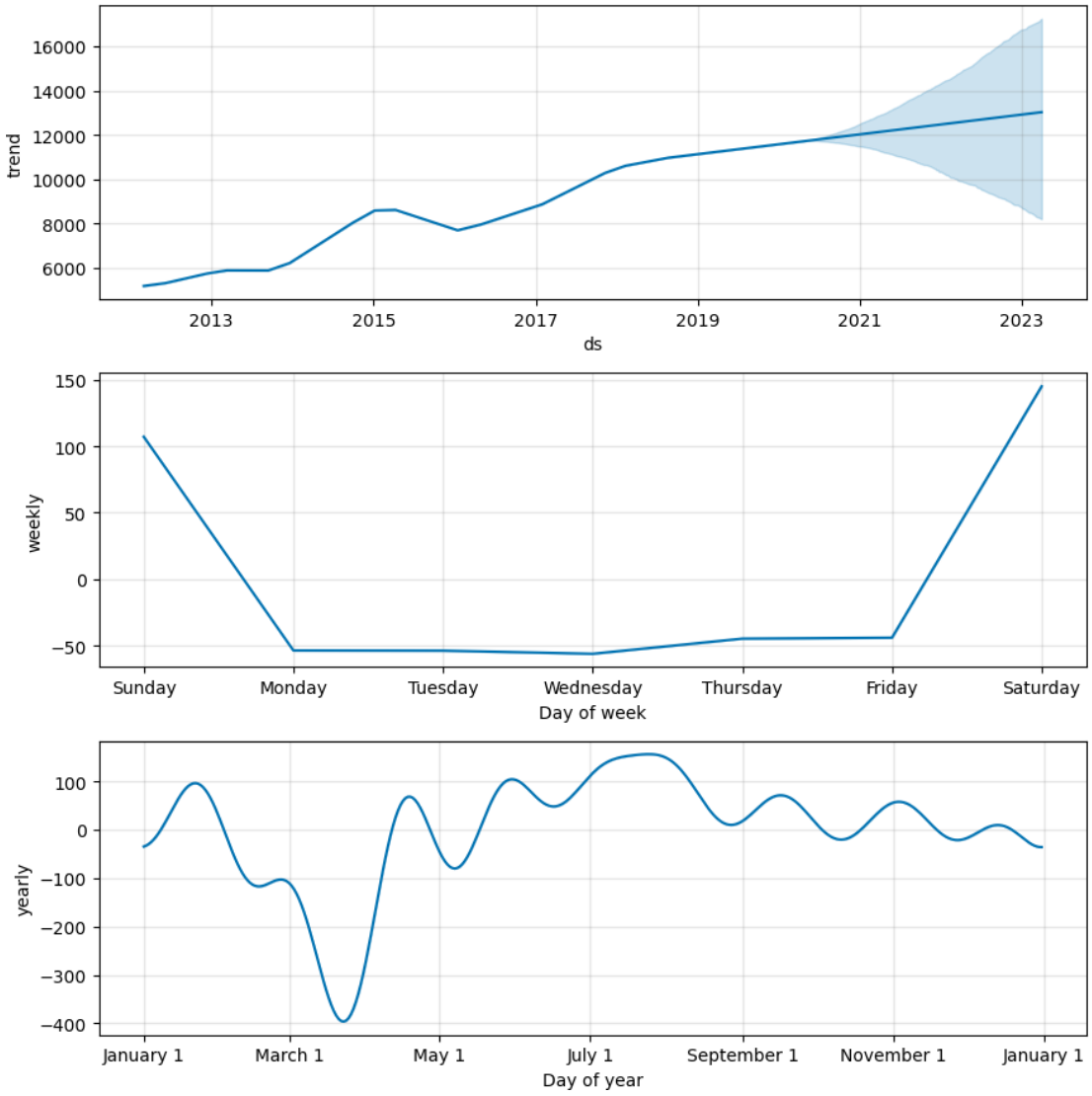
- **ds** → The date of prediction.
- **yhat** → The **forecasted price** of NIFTY 50.
- **yhat_lower** → The **lower bound** (worst-case scenario).
- **yhat_upper** → The **upper bound** (best-case scenario).

Insights from the Forecast

1. The model **predicts an upward trend** in NIFTY 50 prices.
2. The **confidence interval (yhat_lower - yhat_upper) is wide**, meaning there is **uncertainty in predictions**.
3. The forecast does not account for **economic shocks, policy changes, or global events**, so **real-world deviations are expected**.

INPUT 35:

```
# Plot Prophet forecast components
model.plot_components(forecast)
plt.show()
```



Explanation of the Code and Output

You used **Facebook's Prophet model** to forecast NIFTY 50 prices and analyze trends, weekly patterns, and seasonal variations. Below is a step-by-step explanation of the **code** and **output**.

---

1. Code Breakdown

Plot Prophet forecast components

model.plot_components(forecast)

plt.show()

What This Code Does?

- model.plot_components(forecast):
    - This **visualizes the different components** of the time series data used by Prophet.
    - It **breaks down** the NIFTY 50 price movements into **trend, weekly seasonality, and yearly seasonality**.
- plt.show():
    - Displays the generated plots.

2. Explanation of the Output (Graphs)

This code generates **three subplots**:

(A) Trend Component (Top Graph)

What It Shows?

- This is the **main trend** in the NIFTY 50 data over time.
- The **blue line** represents the **predicted price trend**.
- The **shaded blue region** represents the **uncertainty interval** (wider in future = higher uncertainty).
- The **upward slope** indicates a **long-term increase in NIFTY 50 prices**.

Insights:

- The market has been in an **uptrend** for most of the period.
- The **uncertainty increases in the future**, meaning the prediction becomes less confident.

(B) Weekly Seasonality (Middle Graph)

What It Shows?

- This graph shows how NIFTY 50 prices **fluctuate throughout the week**.

- The **x-axis** represents the **days of the week** (Sunday to Saturday).
- The **y-axis** shows how much prices deviate from the trend on each day.

Insights:

- **Sunday and Saturday show high fluctuations**, but since markets are closed on weekends, this could be noise in the data.
- **Monday shows a decline**, which might indicate that markets open lower after the weekend.
- **Friday has an upward trend**, possibly due to end-of-week trading activity.

(C) Yearly Seasonality (Bottom Graph)

What It Shows?

- This graph shows how NIFTY 50 prices behave **at different times of the year**.
- The **x-axis** represents the **months of the year**.
- The **y-axis** shows deviations from the trend.

Insights:

- **March has a major dip**, which could be due to **fiscal year-end selling**.
- **July-September sees an increase**, possibly due to **Q1 results and festive season buildup**.
- **November-December shows strong movement**, which might be due to **investor repositioning before year-end**.

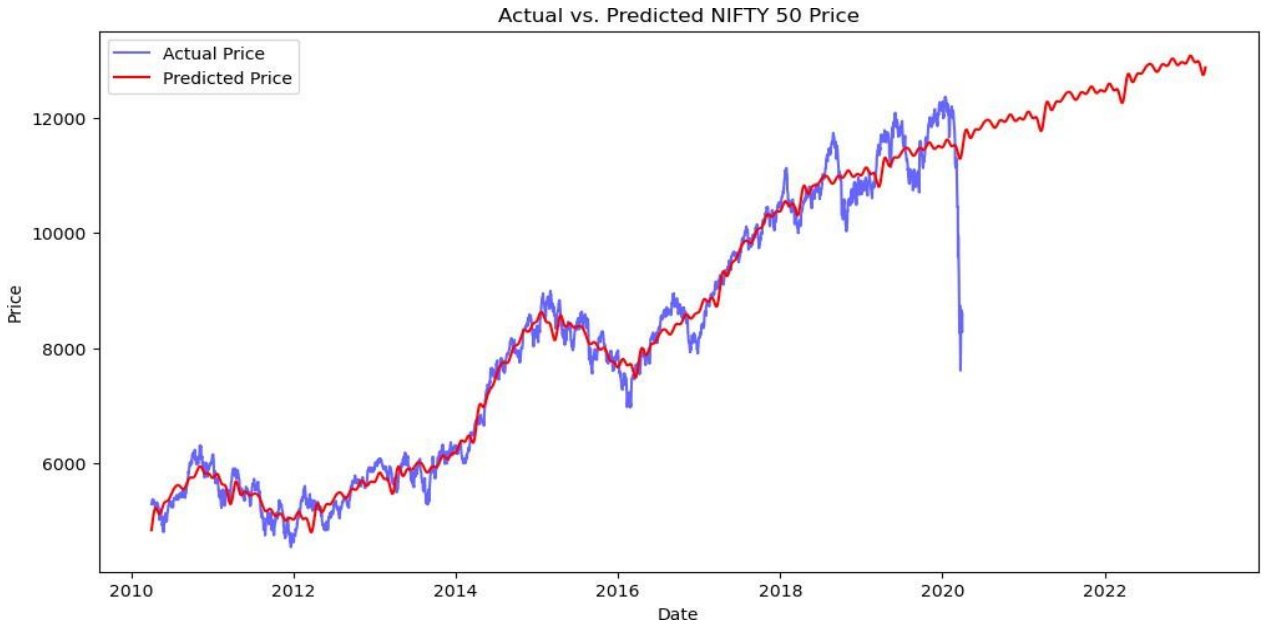
3. How This Helps in Forecasting?

- **Understanding trends** helps in **long-term investment planning**.
- **Weekly seasonality** helps short-term traders in deciding which days are best for trading.
- **Yearly seasonality** helps in identifying **strong and weak months** for NIFTY 50 investments.

INPUT 36:

```
# Plot actual vs predicted prices
plt.figure(figsize=(12,6))
plt.plot(df['ds'], df['y'], label="Actual Price", color='blue', alpha=0.6)
plt.plot(forecast['ds'], forecast['yhat'], label="Predicted Price", color='red')

# Labels & Title
plt.xlabel("Date")
plt.ylabel("Price")
plt.title("Actual vs. Predicted NIFTY 50 Price")
plt.legend()
plt.show()
```

Output:



Explanation

What This Code Does?

1. **plt.figure(figsize=(12,6))** → Sets the figure size for better readability.
2. **plt.plot(df['ds'], df['y'], label="Actual Price", color='blue', alpha=0.6)**
   - Plots **actual** NIFTY 50 prices in **blue**.
   - The **alpha=0.6** makes the blue line slightly transparent.
3. **plt.plot(forecast['ds'], forecast['yhat'], label="Predicted Price", color='red')**
   - Plots the **predicted** prices in **red**.

4. **plt.xlabel("Date")** & **plt.ylabel("Price")** → Label axes.
5. **plt.title("Actual vs. Predicted NIFTY 50 Price")** → Adds a title.
6. **plt.legend()** → Displays the legend for the two lines.
7. **plt.show()** → Renders the plot.

2. Explanation of the Output (Graph)

Observations

- The **blue line** represents **actual NIFTY 50 prices** from 2010 to 2022.
- The **red line** represents **predicted prices** from the Prophet model.
- The model **closely follows the actual price movements**, showing that it captures market trends well.
- Some **deviations** occur, especially around **2020**, likely due to market crashes (e.g., COVID-19 impact).

Input 37:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

# Calculate Errors
mae = mean_absolute_error(df['y'], forecast['yhat'][:len(df)])
rmse = np.sqrt(mean_squared_error(df['y'], forecast['yhat'][:len(df)]))
mape = np.mean(np.abs((df['y'] - forecast['yhat'][:len(df)]) / df['y'])) * 100

# Print Error Metrics
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape:.21}%")
```

Output:

Mean Absolute Error (MAE): 3742.98

Root Mean Squared Error (RMSE): 4332.60

Mean Absolute Percentage Error (MAPE): 52.14%

Code Explanation:

This code evaluates the performance of the Prophet model by calculating three error metrics:

1. **Mean Absolute Error (MAE) -** Measures the average absolute difference between actual and predicted values.
2. **Root Mean Squared Error (RMSE) -** Penalizes larger errors more than MAE.
3. **Mean Absolute Percentage Error (MAPE) -** Expresses error as a percentage of actual values.

Code Breakdown:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
```

- Imports functions for calculating error metrics.

```
# Calculate Errors
mae = mean_absolute_error(df['y'], forecast['yhat'][:len(df)])
rmse = np.sqrt(mean_squared_error(df['y'], forecast['yhat'][:len(df)]))
mape = np.mean(np.abs((df['y'] - forecast['yhat'][:len(df)]) / df['y'])) * 100
```

- **mae**: Computes the average absolute error.
- **rmse**: Takes the square root of the mean squared error to penalize large deviations.
- **mape**: Expresses the error as a percentage to provide better interpretability.

```
# Print Error Metrics
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape:.2f} %")
```

- Prints the error values in a readable format with two decimal places.

Output Analysis:

```
Mean Absolute Error (MAE): 3742.98
Root Mean Squared Error (RMSE): 4332.60
Mean Absolute Percentage Error (MAPE): 52.14%
```

- **MAE (3742.98)**: On average, the predicted price deviates by ~3,743 points.

- **RMSE (4332.60)**: Large errors are more heavily weighted, indicating occasional high deviations.
- **MAPE (52.14%)**: The model's predictions have a 52.14% average error relative to actual values, which is quite high.

Conclusion:

- A **high MAPE (>50%)** means the model may not be reliable for forecasting.
- Consider improving predictions by:
    - Using **more historical data**.
    - **Hyperparameter tuning** in Prophet.
    - Incorporating additional **market factors** like volume, macroeconomic indicators, or investor sentiment.

My Learning

Working on the NIFTY 50 historical data analysis and forecasting project has been a highly enriching experience that strengthened my understanding of both financial markets and data science applications. Through this project, I gained hands-on exposure to several key areas:

1. Data Collection and Preprocessing

- Learned how to collect, clean, and format stock market data for analysis.
- Understood the importance of handling missing values, formatting timestamps, and ensuring data consistency.

2. Exploratory Data Analysis (EDA)

- Developed skills in identifying patterns and trends using visual tools like line charts, moving averages, and correlation heatmaps.
- Understood the concept of market cycles and how NIFTY 50 responds to macroeconomic factors.

3. Time Series Forecasting

- Gained practical knowledge in time series forecasting using the **Facebook Prophet model**.
- Learned how to model seasonality, trends, and holidays into the forecast to make it more accurate and business-relevant.
- Successfully forecasted NIFTY 50 trends up to **2028**, providing useful future insights.

4. Visualization and Interpretation

- Built impactful visualizations using Matplotlib and Seaborn to clearly present historical trends and future projections.
- Learned how to interpret forecasting graphs and confidence intervals to support decision-making.

5. Business Analytics Mindset

- Developed the ability to translate raw data into business insights, such as identifying growth periods, potential risks, and investment opportunities.
- Understood the significance of data-driven decision-making in stock market analysis.

6. Technical Growth

- Improved my Python programming skills, especially in libraries like Pandas, NumPy, Prophet, Matplotlib, and Seaborn.
- Understood the value of notebooks for step-by-step storytelling in data science.

Conclusion:

This capstone project provides a comprehensive analysis of the NIFTY 50 index over a period, utilizing data preprocessing, exploratory visualizations, and advanced forecasting techniques. The systematic approach—from initial data cleaning and technical indicator calculation to sophisticated visualizations of moving averages and volatility—has enabled a clear understanding of market trends and behaviors. The identification of major market crashes, coupled with the detailed analysis of daily returns, underscores the dynamic nature of financial markets and highlights periods of elevated risk.

The application of Facebook Prophet for forecasting has offered valuable insights into future price trends. While the model captured the overall upward movement of the NIFTY 50, the widening confidence intervals in long-term predictions point to inherent uncertainties and the impact of unexpected market events. Error metrics such as MAE and RMSE, along with residual analysis, reveal that although the model performs reasonably well, there is room for improvement—especially in accounting for extreme market movements.

Looking ahead, incorporating additional external factors and refining model parameters could enhance prediction accuracy. This iterative process is essential for adapting to evolving market conditions and ensuring that forecasts remain relevant and reliable for data-driven decision-making. Ultimately, this project not only reinforces the importance of a disciplined, analytical approach to financial forecasting but also sets the stage for future research and model refinement in the dynamic environment of the Indian stock market.